Completeness- and Update-Tailored Database Design

Zhuoxing Zhang, Sebastian Link
The University of Auckland, Auckland, New Zealand
{zhuoxing.zhang, s.link}@auckland.ac.nz

***Abstract*—Normalization judges a relational design by the data redundancy it admits, not by how often the application updates the redundant values. Yet every update of a value that a business rule determines rewrites all its redundant occurrences, and a few rules receive most of the updates. Hence two designs that no classical normal form separates can differ without bound in what one workload costs them. Redundancy has a second cost on the incomplete relations of operational databases, whose records are completed as the business process advances. Filling in a missing value can surface a conflict that earlier updates had hidden behind it. When the new value prevails, repairing the conflict rewrites the same redundant occurrences, so a frequently updated rule stored redundantly is paid for twice. We therefore initiate logical design that takes update frequencies as a declared requirement, next to the completeness requirements that decide on which tuples each rule applies. The heat of a schema is the total update frequency of the non-key functional dependencies it stores. Bounding heat yields a family of normal forms from E-BCNF, the completeness-tailored BCNF, at level zero to E-3NF at the top. Heat is also monotone in the attribute set, which lets our synthesis algorithm return a lossless, dependency-preserving E-3NF decomposition whose hottest subschema is as cool as in any such decomposition. Experiments on a delivery schema show that the update frequencies, and not the structure, decide which design is cheaper, and that choosing by structure can cost more than an order of magnitude. On twelve real-world incomplete relations, our synthesis cools the hottest subschema on about half of them at little cost in structure, and on a real release window its design runs the rule-governed updates fastest.**

***Index Terms*—database design, functional dependencies, incomplete data, normalization, update workloads**

I. Introduction

Business rules enter a relational schema as functional dependencies (FDs), and a decomposition decides how each of them is stored. A rule $X \to A$ stored in a subschema of which X is a key keeps one occurrence per X-value, and an update of its A-value writes one row. Stored as a non-key FD, the rule is enforced by a trigger rather than a key [1], its A-value is repeated in every tuple with the same X-value, and an update rewrites this whole group. The cost of maintaining a rule therefore depends on two quantities: how often the application updates the values the rule governs, and how many redundant occurrences each update must touch. The first is set by the application. Like the access frequencies of records [2]–[4], it is skewed: a few rules receive most of the updates while the majority stays cold. Normal forms address only the second. Hence a design chosen on structure alone can store the hottest rule as a non-key FD, where every routine update rewrites a group instead of one row. This paper makes update frequencies a declared requirement of logical design.

Practitioners state these frequencies before the schema exists. The specification of TPC-C, for example, requires at least 43% of its transactions to be payments, each of which updates the balances of a warehouse, a district, and a customer, while no transaction updates a name or an address [5, Cls. 2 and 5.2.3]. Such frequencies have driven physical design for four decades [6], [7], and the design advisors of commercial systems take a workload of statements and their frequencies as input [8], [9]. Physical design, however, leaves the redundancy of the schema in place, so it can speed up an update but not reduce the rows it rewrites. In logical design, frequencies have entered cost models of how far to normalize [10], [11], but not the normal forms and synthesis algorithms that settle the schema, whose criteria are structural. *Structure-optimal* synthesis ranks 3NF decompositions by the number of non-key FDs in a cover and then by the number of minimal keys [12]. Yet it counts the non-key FDs without asking how often each one is updated. Cardinality-bounded design [13], [14] bounds the other quantity, how many occurrences an update touches, but likewise ignores how often updates occur.

Why incompleteness matters. Operational databases hold incomplete relations [15], whose records are created incomplete and completed as the business process advances. TPC-C enters every new order without a carrier and every order line without a delivery date, and its Delivery transaction, at least 4% of all transactions, fills both in [5, Cls. 2.4.2, 2.7.4, and 5.2.3]. TPC-C even constrains which values are missing: an order line lacks a delivery date if and only if its order lacks a carrier [5, Cl. 3.3.2.7]. Missing values are thus part of the specification, and filling them in is part of the workload. No design theory over complete relations can express such updates. Over incomplete relations, completeness-tailored design embeds the completeness requirements of an application in the declaration of its constraints: an embedded FD or uniqueness constraint (eFD, eUC) names a set of attributes, its *embedding*, on which tuples must be complete for the rule to apply [16], [17]. E-BCNF and E-3NF consolidate the completeness requirements of an application in a design scope E and grade a design by the data redundancy it admits on the tuples complete on E [17], but not by how often that redundancy is updated.

Like completeness, update frequencies are a requirement of

Full version. The full version of this paper, with an appendix that gives all proofs, is in the repository https://github.com/zhuoxingzhang/completeness-update-tailored-design.

TABLE I: A delivery snapshot r: the scope r^E above the line, the redundant occurrences of the hot rule $z \to c$ in bold, and the pending tuple t_6 below it, outside the scope.

	b(ranch)	c(ourier)	d(ay)	g(ps)	r(ound)	v(an)	z(one)
t_1	North	**Maggie**	Mon	Telco-A	R1	V7	Z1
t_2	South	**Maggie**	Mon	Telco-B	R2	V9	Z1
t_3	North	**Maggie**	Wed	Telco-A	R1	V7	Z1
t_4	North	Maggie	Tue	Telco-A	R1	V7	Z2
t_5	North	Omar	Mon	Telco-B	R2	V4	Z3
t_6	East	Ravi	Tue	$\perp$	R3	V8	Z1

the application, and missing values change what updates cost in three ways. Firstly, the frequency and the redundancy of a rule are both confined to its *scope*, the tuples complete on its embedding, and a *pending* tuple outside the scope needs no maintenance until it enters. Secondly, besides changing a stored value, which we call a *refresh*, an update may fill in a missing value, which we call a *completion*. A tuple that a completion brings into the scope of a rule may disagree with the tuples already there, in a conflict that its null marker $\perp$ had hidden. A refresh changes values only within the scope and leaves behind the pending tuples that agreed with the old value, so a hot rule creates such conflicts at the rate at which it is refreshed. When the completed value prevails, the redundancy that decides the cost of a refresh also decides the cost of resolving that conflict, so a hot rule stored as a non-key FD is paid for twice. Thirdly, the scope grows as the data completes [17], and with it the groups of every rule, so a hot rule stored as a non-key FD grows more expensive over time. A design for incomplete data must therefore meet both requirements at once, which neither completeness-tailored design nor frequency-driven physical design does.

Cooling hot rules by decomposition. Of the two quantities that price a rule, the redundancy is the one the design controls [1], [18]. Not every schema has a decomposition into BCNF that is both lossless and dependency-preserving [19], so some rules remain non-key FDs, and the data decide how often the scope repeats their values. Which rules those are is a choice of the decomposition, and we make it by frequency: the rules left non-key must be the ones the application rarely updates. This is illustrated as follows.

Example 1 (Delivery zones)**.** A courier service records deliveries over schema $R = \{branch, courier, day, gps, round, van, zone\}$, abbreviated $\{b, c, d, g, r, v, z\}$. The GPS network g a van reports through is registered only after its unit first reports in, so a tuple can stay pending with $g = \perp$. The application declares each constraint with the completeness requirement under which it applies, and consolidates its requirements in the design scope $E = R$ of fully complete tuples. The declared set Σ consists of the eFD $(cgz; z \to c)$ and, each with embedding equal to its attributes, the rules $bcd \to z$, $bc \to v$, $bv \to c$, $cv \to r$, $rz \to b$, and $v \to g$. A zone has one courier on duty among the deliveries whose van has reported in, and a van has one courier within its

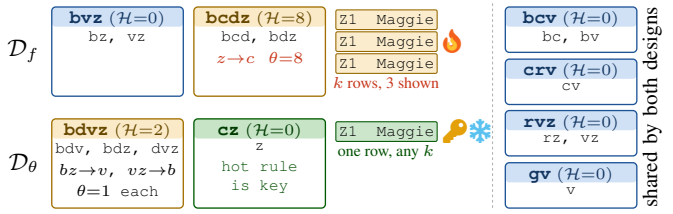


Fig. 1: Two E-3NF decompositions of the delivery schema of Ex. 1, agreeing on four subschemata and parting on two: $\mathcal{D}_f$, returned by structure-optimal synthesis, and $\mathcal{D}_\theta$, returned by ours. Each card is one subschema with its minimal keys and heat $\mathcal{H}$; orange cards maintain non-key FDs.

branch. Replacing a courier refreshes both $z \to c$ and $bv \to c$. Couriers are reassigned whenever shifts change or a driver drops out, while the van, round, or branch of a delivery is corrected only on occasion. The application therefore declares the update frequency, or *heat*, of the courier as 8 and of every other attribute as 1. Table I shows a snapshot. Fig. 1 shows the decompositions $\mathcal{D}_f$ and $\mathcal{D}_\theta$ that structure-optimal synthesis and ours return (Ex. 24), both lossless and dependency-preserving over E. Each has five subschemata in E-BCNF and one critical subschema, in E-3NF but not in E-BCNF. The critical subschema stores one non-key FD in $\mathcal{D}_f$ and two in $\mathcal{D}_\theta$, so structure even prefers $\mathcal{D}_f$. Under the workload they differ sharply. Indeed, $\mathcal{D}_f$ stores the hot rule as a non-key FD on $bcdz$, which holds a zone in k rows, one per branch and day, and repeats its courier in each. A reassignment thus rewrites all k rows, whereas $\mathcal{D}_\theta$ makes the zone a key of the two-attribute subschema cz and rewrites one row there. A van swap, which refreshes the van of a courier, is the mirror image. It rewrites one row per zone of the courier in bvz on $\mathcal{D}_f$ and one row per day in $bdvz$ on $\mathcal{D}_\theta$. Hence the update frequencies, and not the structure, decide which design is cheaper.

The same redundancy prices the repair of a conflict.

Example 2 (Completions surface latent conflicts)**.** The pending tuple t_6 of Table I claims courier *Ravi* for zone *Z1*, whose scope tuples t_1–t_3 all carry *Maggie*. The disagreement is invisible while g is missing. When the unit of the van first reports in, its network is registered and t_6 enters the scope. The application lets the newcomer prevail: the zone must be reassigned to *Ravi*, and with it zone *Z2*, since t_4 shares branch and van with t_1 ($bv \to c$). On $\mathcal{D}_f$ the repair rewrites the k rows of *Z1* in $bcdz$ and those of *Z2*, four in the snapshot, and on $\mathcal{D}_\theta$ one row per zone in the key subschema cz, two at any k. On $\mathcal{D}_f$ the hot rule is thus paid for twice, by every reassignment and every repair.

Contributions. We develop completeness- and update-tailored database design, the first framework that tailors schemata simultaneously to the completeness requirements and the update frequencies of an application. Over complete relations every scope is the whole relation, and the framework tailors 3NF synthesis to update frequencies alone.

1) *Conceptually*, we identify the update workload as a second declared requirement of logical design, next to completeness, and show that the frequencies of updates issued against different rules and embeddings add up per attribute on the design scope. We also classify the updates of incomplete data and show that retractions, which replace a value by $\perp$, need no maintenance, while completions alone surface the conflicts that refreshes leave hidden outside the scope. When the completed value prevails, repairing such a conflict rewrites the same occurrences as a refresh.
2) *Computationally*, we price a schema by its heat, the total update frequency of the non-key FDs it stores, and bound heat by a family of normal forms from E-BCNF, the anomaly-free level zero, to E-3NF. Heat is monotone in the attribute set and separates designs with equally many subschemata by an unbounded factor. Recognizing a level is NP-hard, as recognizing E-3NF is, so the frequencies add no hardness of their own.
3) *Algorithmically*, we give a synthesis algorithm that returns a lossless, dependency-preserving E-3NF decomposition over the scope whose maximal subschema heat is minimal among all such decompositions, in time polynomial in the number of implied FDs with minimal left-hand sides. Whether frequencies can separate a given design from this minimum is decided by implication tests before any frequency is declared.
4) *Empirically*, we show on a delivery schema that the update frequencies, and not the structure, decide which design is cheaper over a maintenance window, that choosing by structure can cost more than an order of magnitude, and that only a declaration by the measured rates lets synthesis reach the cheaper design under all three workload profiles. On the schemata of twelve real-world incomplete relations, our synthesis is never hotter than the baselines and cools the hottest subschema on about half of them at little cost in structure. On a real release window, with constraints and heat mined before the release, its design runs the updates that the mined FDs govern fastest.

Organization. Sec. II recalls completeness-tailored design, Sec. III develops the update taxonomy, heat, and the normal forms, and Secs. IV and V present synthesis and completions. Sec. VI reports experiments, Sec. VII discusses related work, and Sec. VIII concludes. The appendix of the full version gives all proofs.

II. Preliminaries

We recall incomplete relations, embedded constraints, and the completeness-tailored normal forms of Wei and Link [17]. A *relation* r over a finite set R of attributes is a finite set of tuples t that assign each $a \in R$ a value $t(a) \in \mathrm{dom}(a) \cup \{\perp\}$, where the null marker $\perp$ is read as *no information* [20], [21]. A tuple t is X*-total* if $t(a) \neq \perp$ for all $a \in X$.

An *embedded functional dependency* (eFD) over R is an expression $(E; X \to Y)$ with $XY \subseteq E \subseteq R$, and an *embedded uniqueness constraint* (eUC) is an expression $(E; X)$ with $X \subseteq E \subseteq R$ [16], [17]. The *scope* of an embedding E in r is $r^E = \{t \in r \mid t \text{ is } E\text{-total}\}$. A relation r satisfies the eFD $(E; X \to Y)$ if every pair of tuples in r^E that agrees on X agrees on Y, and it satisfies the eUC $(E; X)$ if no two distinct tuples in r^E agree on X. Over total relations every scope is the whole relation, and eFDs and eUCs are FDs and UCs.

An application declares a set Σ of eFDs and eUCs and consolidates its completeness requirements in a single *design scope* $E \subseteq R$ [17]. The *reduct* of Σ to E is the set $\Sigma[E]$ of ordinary FDs $X \to Y$ and UCs X arising from members of Σ whose embedding is contained in E. On the scope, where every tuple is E-total, these members hold exactly when $\Sigma[E]$ holds classically, so design over the scope is classical design for $\Sigma[E]$ [17]. A schema (R, Σ) with design scope E is in *E-BCNF* (*E-3NF*) when $(R, \Sigma[E])$ is in BCNF (3NF), and E-losslessness and E-dependency-preservation are losslessness and dependency-preservation with respect to r^E [17]. For $S \subseteq R$, the constraints Σ_S of the subschema S are the FDs over S that $\Sigma[E]$ implies, and the subschema is in E-BCNF (E-3NF) when (S, Σ_S) is in BCNF (3NF). Since S fixes Σ_S, we name a subschema by its attribute set alone, and write (R_i, Σ_i) for a subschema R_i with $\Sigma_i = \Sigma_{R_i}$.

A design stores r^E projected onto its subschemata and the remaining tuples in a full-width *remainder* [17]. The subschemata enforce $\Sigma[E]$ on the scope, and an eFD whose embedding is a proper subset of E is checked on a tuple of the remainder once that tuple becomes E-total. E-BCNF and E-3NF grade such a design by the redundancy it admits on the scope. Sec. III prices that redundancy by how often the application updates it.

III. Updates, Heat, and Tailored Normal Forms

Throughout, fix a schema (R, Σ) with design scope E. This section classifies the updates of incomplete relations, attaches declared frequencies to the constraints they refresh, defines the heat of a schema, bounds it by a family of normal forms justified by update anomalies, and settles their complexity.

A. A Taxonomy of Updates over Incomplete Data

Updates over incomplete relations differ from updates over complete ones in one respect: a value can move into or out of $\perp$, and thereby a tuple can enter or leave a scope.

Definition 3 (Update taxonomy)**.** Let r be a relation over R, $t \in r$, $a \in R$, and let t' result from t by replacing $t(a)$ with v. The replacement is a *refresh* if $t(a) \neq \perp$ and $v \neq \perp$, a *completion* if $t(a) = \perp$ and $v \neq \perp$, and a *retraction* if $t(a) \neq \perp$ and $v = \perp$.

Retractions need no constraint maintenance, since they only shrink scopes. Indeed, a retracted tuple either leaves a scope or keeps its projection onto the embedding, so every pair of tuples that a constraint still sees is a pair it saw before. Completions, conversely, grow scopes and can therefore cause violations, whose cost Sec. V analyzes. Inserts reduce to these two cases. An insert of an E-total tuple enters the scope and costs what a completion costs. An insert of a tuple with a missing value in

E lands outside the scope and, like a retraction or a deletion, needs no maintenance until a completion moves it in. The rest of this section studies refreshes, the routine updates.

B. Update Modes and Heat

Applications refresh the values their business rules determine, so we declare update behavior on the constraints, as completeness requirements are declared on them [17].

Definition 4 (Update mode)**.** An *update mode* over R is an expression $\mu = (E'; X \to A)$ with $XA \subseteq E' \subseteq R$. An application of μ to r picks an X-total condition tuple x and a value $v \in \mathrm{dom}(A)$, and refreshes $t(A) := v$ for every $t \in r^{E'}$ with $t[X] = x[X]$.

The tuples refreshed form the *X-group* of x in $r^{E'}$, the tuples with matching values on X. Every declared eFD $(E'; X \to Y) \in \Sigma$ induces one mode per attribute of Y, and M_Σ denotes the set of these modes. How often their applications refresh each attribute is the second requirement the application declares.

Definition 5 (Heat map)**.** A *heat map* is a function θ assigning each attribute that a mode of M_Σ refreshes a positive heat, the frequency with which the application refreshes it on the scope through any of these modes, on a common scale.

Like the design scope E, the heat map is stated by the application and not read off an instance. Only the ratios of heats matter, since schemata are compared by sums of heats (Def. 9 below). An attribute without a stated frequency has the default heat 1, the least a declaration can state, so every non-key FD costs at least 1. A declared mode names one rule, but several rules of the reduct can determine one attribute, as $z \to c$ and $bv \to c$ determine the courier, and a refresh has to keep all of them satisfied.

Lemma 6 (Co-refreshed rules)**.** *Let r^E satisfy $\Sigma[E]$, let v be* fresh *in that it occurs in no a-value of r^E, and let r' arise from r^E by rewriting the a-value of the tuples of a set T to v. Then r' satisfies every non-trivial FD $X \to a$ that $\Sigma[E]$ implies if and only if T is a union of* components *of a, the classes of the finest partition of r^E in which every X-group of every such FD lies within one class.*

Indeed, the rewrite leaves every X-group in place, and since v is fresh, only the tuples of T carry it afterwards. The FD $X \to a$ then holds after the rewrite exactly when no X-group has tuples both inside and outside T. Hence an application of a mode rewrites the component of its group, whatever rule it is issued against, and moves a whole group of every rule that determines the attribute. A value that is not fresh rewrites the same component and is admissible only if it violates no rule whose determinant contains a.

Example 7 (The component of a reassignment)**.** In Table I, zone *Z1* is reassigned to a new courier Ada. The mode $z \to c$ names t_1–t_3, but t_1 and t_4 share branch $North$ and van $V7$, so $bv \to c$ carries t_4 along and the component is t_1–t_4: zone *Z2* moves with *Z1*, since its courier drives the same van from the same branch. Rewriting t_1–t_3 alone violates $bv \to c$, so both courier rules are refreshed. Reassigning *Z1* to $Omar$ instead, a value that is not fresh, rewrites the same component but violates $bc \to v$, as t_1 and t_5 then agree on branch and courier but not on van.

Several modes can refresh one attribute, through different rules and under different embeddings.

Lemma 8 (Heat aggregation)**.** *Let $\mu_1, \ldots, \mu_m$ be the modes in M_Σ that refresh the attribute A and have embeddings contained in E. Every application of any μ_i that refreshes a tuple of the scope rewrites there a whole group of every non-trivial FD $X \to A$ that $\Sigma[E]$ implies, whatever the mode's embedding. Hence every such FD is refreshed on the scope as often as A is refreshed there through $\mu_1, \ldots, \mu_m$ together.*

A design is therefore priced by one frequency per attribute, whatever rules and embeddings the updates are issued against. From now on every declared mode is embedded within E, like the rules of the reduct. Refreshes of pending tuples in the remainder cost the same on every design and are left out.

Applying a mode to a design. A design is maintained subschema by subschema, each under its own constraints, as in the update studies of structure-optimal design [1], [12]. On a decomposition, an application of a mode $X \to A$ rewrites the A-values of the component that contains its X-group (Lem. 6), and every subschema is then kept equal to the projection of the refreshed scope. A subschema that contains XA rewrites the rows of the group, one row where X is a key of it. A subschema that contains A but not all of X cannot locate the group and replaces the projections of the component's tuples, one row per distinct projection, a cost that Sec. IV returns to.

C. The Heat of a Schema

A refresh rewrites a group, rather than one row, only on a subschema that stores its rule as a non-key FD. Dependency preservation over the scope forces a design to store an FD this way whenever no cover makes it a key dependency [17], [22]. Which FDs a schema stores as non-key is decided by its attribute set: structure-optimal design counts how few of them need a trigger of their own [12], and we count how often all of them are refreshed.

The rules a design can store are the members of the *atomic closure* $\mathcal{A}$ of $\Sigma[E]$, the non-trivial FDs $X \to a$ that $\Sigma[E]$ implies and whose determinant X is minimal [23], [24], so heat is assigned to them. The *aggregated map* θ_E gives each member $X \to a$ of $\mathcal{A}$ the heat of a, $\theta_E(X \to a) = \theta(a)$, since every rule that determines a is refreshed as often as a (Lem. 8).

Definition 9 (Heat of a schema)**.** The *heat* of a schema (R_i, Σ_i) with atomic closure $\mathcal{A}_i$ is $\mathcal{H}_\theta(R_i) = \sum_{X \to a} \theta_E(X \to a)$, summed over the members $X \to a$ of $\mathcal{A}_i$ whose determinant X is not a superkey of R_i.

That is, a schema is charged the heat of a once for each of its non-key FDs $X \to a$ with a minimal determinant X, since

every refresh of a moves a group of each of them (Lem. 6). Keys cost no heat, since in a subschema of which X is a key the group is one row and its refresh is a plain write. The group sizes that multiply a frequency belong to the instance, which does not exist when the schema is designed and grows as the scope fills (Sec. V). A design is chosen once, for every instance, so heat weighs each rule by its declared frequency.

Example 10 (Heat of the delivery subschemata)**.** The atomic closure of the delivery reduct of Ex. 1 adds eleven implied FDs to the declared rules, among them $bz \to v$, $vz \to b$, and $bdv \to z$. The minimal keys of R are bcd, bdv, bdz, drz, and dvz. The undecomposed schema R stores 16 non-key FDs, the two that determine the courier at its heat 8 and the others at heat 1, so $\mathcal{H}_\theta(R) = 30$. Subschema cz of $\mathcal{D}_\theta$ has the key z and no non-key FD, so its heat is 0. Subschema $bcdz$ of $\mathcal{D}_f$ has the keys bcd and bdz and the non-key FD $z \to c$, so $\mathcal{H}_\theta(bcdz) = 8$. Subschema $bdvz$ of $\mathcal{D}_\theta$ has the non-key FDs $bz \to v$ and $vz \to b$, so $\mathcal{H}_\theta(bdvz) = 2$.

The two critical subschemata, $bcdz$ and $bdvz$, which are in E-3NF but not in E-BCNF, stay below the heat of R. This is no coincidence: adding attributes can only add non-key FDs.

Proposition 11 (Monotonicity)**.** *If $S \subseteq S'$ are schemata over R, then $\mathcal{H}_\theta(S) \leq \mathcal{H}_\theta(S')$.*

D. A Family of Tailored Normal Forms

Bounding heat yields a family of normal forms.

Definition 12 ((h, E)-tailored normal form)**.** Let $h \geq 0$ be a *level* of heat. A schema (R_i, Σ_i) is in (h, E)-*tailored normal form*, or (h, E)-TNF, if it is in E-3NF and $\mathcal{H}_\theta(R_i) \leq h$. A decomposition is in (h, E)-TNF if all its subschemata are. Its *heat* is the maximum subschema heat.

A decomposition is weighed by its hottest subschema, not by the sum over its subschemata. A level is thus, like every normal form, a guarantee on every subschema: it bounds the refreshes each one absorbs, while a bound on the sum lets a single subschema absorb them all. By Prop. 11 no subschema is hotter than R, so decomposing never raises the level. Synthesis, which examines the hottest critical schemata first, drives the level to the least value any cover allows (Sec. IV).

The extremes of the family are the classical normal forms.

Proposition 13 (E-BCNF as level zero)**.** *A schema has heat 0 if and only if it is in E-BCNF, that is, in $(0, E)$-TNF.*

The levels are nested and topped by E-3NF.

Proposition 14 (Strict hierarchy)**.** *For $h \leq h'$, every schema in (h, E)-TNF is in (h', E)-TNF, and $(\mathcal{H}_{\text{tot}}, E)$-TNF, with the per-schema bound $\mathcal{H}_{\text{tot}} = \sum_{\sigma \in \mathcal{A}_i} \theta_E(\sigma)$, coincides with E-3NF. Every step is strict for suitable Σ and θ.*

Indeed, $\{a, b, c\}$ with the rules $ab \to c$ and $c \to b$ is in (h, E)-TNF exactly when h is at least the heat of $c \to b$.

Under uniform heat, the family reduces to structure.

Proposition 15 (Uniform heat recovers structure)**.** *If θ_E is constant, the heat of a schema is proportional to its number of non-key atomic FDs, and minimizing decomposition heat minimizes the maximal number of such FDs per subschema.*

Under skew, however, the family separates designs with equally many subschemata, and no structural measure bounds that separation.

Proposition 16 (Unbounded separation)**.** *For every L there are (R, Σ, E, θ) with two E-3NF decompositions with equally many subschemata that are lossless and dependency-preserving over E and whose heats differ by a factor of L. The hotter design stores one non-key FD fewer.*

The running example illustrates Props. 15 and 16.

Example 17 (Levels of the delivery designs)**.** By the heats of Ex. 10, the designs of Fig. 1 sit at levels 8 and 2: $\mathcal{D}_f$ is in $(8, E)$-TNF and in no lower level, and $\mathcal{D}_\theta$ is in $(2, E)$-TNF. Both are in E-3NF and neither is in E-BCNF, so the classical normal forms do not separate them, and the count of non-key FDs, which a uniform declaration measures (Prop. 15), prefers $\mathcal{D}_f$. Declaring the courier at heat $2L$ instead of 8 raises the level of $\mathcal{D}_f$ to $2L$ and leaves that of $\mathcal{D}_\theta$ at 2, which is the separation of Prop. 16 on this schema.

E. Justification by Update Anomalies

Normal forms are justified by the anomalies they characterize [18], [25], [26], and E-BCNF by E-redundancy [17]. We justify the heat hierarchy by the updates it forces. Subschemata generated from the reduct lie within E and store projections of the scope, so their instances are total relations.

Definition 18 (Anomalous application)**.** An application of a mode $X \to A$, with condition x and a fresh value v, is *anomalous* on a schema instance $r_i \models \Sigma_i$ if rewriting the A-value of a single tuple of the X-group of x violates Σ_i, while rewriting the whole group does not.

An anomalous application cannot be confined to one tuple, since satisfaction forces the write onto the whole group.

Proposition 19 (Anomaly boundary)**.** *A schema admits an anomalous application on some schema instance if and only if it is not in E-BCNF.*

Indeed, if $X \to a$ is non-key, two tuples that agree exactly on the closure of X satisfy the constraints, and rewriting a in one of them alone violates $X \to a$. On an E-BCNF schema, every group is a single tuple, so no application is anomalous. The levels above $h = 0$ measure what enforcing the unavoidable anomalies costs. A schema is enforced by declaring its minimal keys as uniqueness constraints and a cover of its non-key FDs as triggers on writes to their attributes [1]. Hence heat counts the refreshes that enforcement leaves to triggers, once per non-key FD a refresh moves, and level h bounds it.

The bound extends to completions. A completion that enters the scope with a fresh conflicting value forces the write pattern

of Def. 18 on every subschema that stores the rule as a non-key FD (Prop. 28). Hence the anomalies a level tolerates are also the repairs that completions can force.

F. Complexity

The framework poses two problems: recognizing the level of a given schema, and deciding whether a design at a given level exists, which at level 0 is the classical question of a lossless, dependency-preserving decomposition into BCNF [19].

Theorem 20 (Determination)**.** *Given a schema with a cover of its projected constraints Σ_i, a heat map, and a bound h, deciding membership in (h, E)-TNF is NP-hard, already under constant heat. If the atomic closure $\mathcal{A}_i$ is computable in polynomial time, the problem is in NP and interreducible with deciding E-3NF. Given (R, Σ), E, a heat map, and h, deciding whether an E-lossless, E-dependency-preserving decomposition into (h, E)-TNF exists is NP-hard, already for $h = 0$ under constant heat.*

The frequencies add no hardness of their own. Under constant heat, a level no lower than $\mathcal{H}_{\text{tot}}$ coincides with E-3NF (Prop. 14), which is already hard to recognize [27]. Level 0 coincides with E-BCNF (Prop. 13), and designing for it is the BCNF question above. Synthesis, however, need not recognize levels: given $\mathcal{A}_i$, heat costs one attribute closure per member.

IV. Heat-Aware Synthesis

We now determine the minimum heat over all E-lossless, E-dependency-preserving, E-3NF decompositions and give a synthesis algorithm that attains it.

A decomposition is E-dependency-preserving exactly when the members of $\mathcal{A}$ within its subschemata imply $\mathcal{A}$. An E-3NF schema that stores an atomic FD $X \to a$ contains Xa, itself in E-3NF [28], [29], so by Prop. 11 it is no cooler than Xa.

Lemma 21 (Selection lower bound)**.** *Let h^* be the minimum of $\max_{X \to a \in F} \mathcal{H}_\theta(Xa)$ over all* covers *$F \subseteq \mathcal{A}$, the subsets that imply $\mathcal{A}$. Every E-lossless, E-dependency-preserving, E-3NF decomposition has maximal subschema heat at least h^*.*

Equivalently, h^* is the least level h at which the members of $\mathcal{A}$ whose schemata have heat at most h imply $\mathcal{A}$, so implication tests alone find it. We compute it on the running example.

Example 22 (The delivery optimum)**.** In Ex. 1, every cover retains $z \to c$, $bc \to v$, $bv \to c$, $cv \to r$, and $v \to g$, whose schemata have heat 0, and retains or derives $bcd \to z$. The cheapest derivation retains $bdv \to z$, with $\mathcal{H}_\theta(bdvz) = 2$, while $\mathcal{H}_\theta(bcdz) = 8$. Hence $h^* = 2$, attained by $\mathcal{D}_\theta$.

To reach h^*, Alg. 1 follows classical synthesis [28], [29] and its structure-optimal refinement, which eliminates redundant critical schemata before redundant BCNF ones [12]. It makes two changes: it runs on the reduct, and it orders the eliminations by the declared frequencies. Starting from the working set $\mathcal{A}_w = \mathcal{A}$, the loop on line 2 examines the members in turn. A member $X \to a$, and with it the schema Xa, is redundant when the other members of $\mathcal{A}_w$ imply it, and the loop then drops it from $\mathcal{A}_w$, while otherwise Xa joins the decomposition. Redundant critical schemata are eliminated in decreasing order of heat rather than of their number of non-key FDs. The heat order attains h^* (Thm. 23): the members whose schemata have heat at most h^* imply $\mathcal{A}$ and are still in $\mathcal{A}_w$ when each hotter critical schema is examined, so each hotter one is found redundant and dropped. Ordering by structure instead can miss h^* by any factor (Prop. 16). Redundant E-BCNF schemata all have heat zero, but every refresh of an attribute rewrites each subschema that contains it. They are therefore eliminated in decreasing order of the refreshes they receive, their *refresh weight* $w_\theta(S) = \sum_{b \in S} \theta(b)$, and so are critical schemata of equal heat. An E-BCNF schema inside a retained critical subschema weighs 0, since line 4 absorbs it. Eliminating the heaviest first keeps the maximal refresh weight of the E-BCNF subschemata as low as the eliminations allow (Thm. 23).

Algorithm 1 TAILOR(R, Σ, E, θ)

Require: schema R, constraints Σ, design scope E, heat map θ
Ensure: E-lossless, E-dependency-preserving, E-3NF decomposition $\mathcal{D}$ of (R, Σ) over E that minimizes the maximal subschema heat

1: $\mathcal{A} \leftarrow$ atomic closure of $\Sigma[E]$ [23], [24]; $\theta_E \leftarrow$ aggregated map of θ on $\mathcal{A}$; $\mathcal{A}_w \leftarrow \mathcal{A}$; $\mathcal{D} \leftarrow \emptyset$
2: **for all** $X \to a \in \mathcal{A}$, critical schemata Xa first, in decreasing order of $\mathcal{H}_\theta(Xa)$, then of $w_\theta(Xa)$, or 0 for an E-BCNF Xa inside a critical subschema in $\mathcal{D}$, ties in the order of $\mathcal{A}$ **do**
3: **if** $\mathcal{A}_w \setminus \{X \to a\} \models X \to a$ **then** $\mathcal{A}_w \leftarrow \mathcal{A}_w \setminus \{X \to a\}$ **else** $\mathcal{D} \leftarrow \mathcal{D} \cup \{(Xa, \Sigma_{Xa})\}$
4: $\mathcal{D} \leftarrow \mathcal{D} \setminus \{(S, \Sigma_S) \mid \exists\, (S', \Sigma_{S'}) \in \mathcal{D}$ with $S \subsetneq S'\}$
5: **if** $S \to R \notin \Sigma[E]^+$ for all $(S, \Sigma_S) \in \mathcal{D}$ **then**
6: $\mathcal{D} \leftarrow \mathcal{D} \cup \{(K, \Sigma_K)\}$ for a minimal key K of $(R, \Sigma[E])$ [30]
7: store r^E in $\mathcal{D}$ and $r \setminus r^E$ in a full-width remainder [17]
8: **Return** $\mathcal{D}$

Theorem 23 (Correctness and optimality)**.** *Alg. 1 returns a decomposition that is E-lossless, E-dependency-preserving, and in E-3NF, and whose maximal subschema heat is h^*, the minimum over all such decompositions (Lem. 21). Subject to that minimum, the maximal refresh weight over the E-BCNF subschemata that line 4 leaves is minimal among the decompositions obtained by eliminating redundant FDs from the set $\mathcal{A}_w$ left once line 3 has examined every critical schema.*

Correctness follows the classical synthesis argument on the scope [29]: the eliminations keep a cover of $\mathcal{A}$ embedded, and every Xa is in E-3NF. Thm. 23 uses heat only through Props. 11 and 13, so it holds for every monotone cost that is zero on E-BCNF schemata. We illustrate Alg. 1 by deriving the designs of Fig. 1.

Example 24 (Synthesis on the delivery reduct)**.** We start with the atomic closure of Ex. 10, whose 18 members generate two critical schemata, both from a rule that determines the zone. The schema $bcdz$, generated by $bcd \to z$, stores the hot rule $z \to c$ and has heat 8, and $bdvz$, generated by $bdv \to z$, stores $bz \to v$ and $vz \to b$ and has heat 2. Line 2 therefore examines $bcd \to z$ first. It is redundant, since $bc \to v$ and

$bdv \to z$ imply it, and is removed. The FD $bdv \to z$ is then not redundant, since the remaining FDs derive only c, g, and r from bdv, so $bdvz$ is added to the decomposition. The loop then examines the E-BCNF schemata by refresh weight, from 10 for bcg, bcr, bcv, and crv down to 0 for bvz, which lies inside $bdvz$. Ties between schemata of equal refresh weight follow the order of $\mathcal{A}$, and on the order we trace, the loop retains bcv, crv, cz, rvz, gv, and bvz, the schemata of the FDs that the remaining ones do not imply. Since $bvz \subseteq bdvz$, line 4 removes bvz, and line 6 adds no key, since $bdvz$ contains the minimal key bdz of R. The final decomposition is $\mathcal{D}_\theta$, of heat $h^* = 2$. Structure-optimal synthesis examines $bdv \to z$ first, as $bdvz$ stores two non-key FDs and $bcdz$ one. This FD is redundant while $bcd \to z$ is present, and its removal leaves $bcd \to z$ irredundant, so $bcdz$ of heat 8 is added instead. Its E-BCNF pass, by numbers of minimal keys, retains the same six schemata, but line 4 now absorbs cz into $bcdz$ and leaves bvz, which no retained subschema contains, so the result is $\mathcal{D}_f$. Other orders of $\mathcal{A}$ retain brz in place of bvz and rvz, and both algorithms then return that variant, with the same critical subschema. On $\mathcal{D}_f$ and $\mathcal{D}_\theta$ the count of non-key FDs and the heat point in opposite directions.

Alg. 1 is efficient once the atomic closure is known. The closure, computed on line 1, can be exponential in the size of Σ and is the only source of exponential cost, as in structure-optimal synthesis [12]. Worst-case exponential time is unavoidable unless P = NP, since the level that Alg. 1 attains decides the design problem of Thm. 20.

Proposition 25 (Cost of synthesis)**.** *Given the atomic closure $\mathcal{A}$, Alg. 1 runs in time polynomial in $|\mathcal{A}|$ and $|R|$.*

Hence h^* is computable in polynomial time from $\mathcal{A}$.

Alg. 1 attains h^* for the declared heat map, while a design chosen by another criterion, such as $\mathcal{D}_f$, can sit at the optimum under one declaration and far from it under another. Implication tests on $\mathcal{A}$ also decide, before any frequency is declared, whether frequencies can separate a given design from the optimum. For a schema S and an attribute b, let $n_S(b)$ count the members of $\mathcal{A}$ inside S with right-hand side b whose determinant is not a superkey of S. A schema S' *dominates* S if $n_S(b) \leq n_{S'}(b)$ for every b. A schema S *stores* b if $n_S(b) > 0$, and a decomposition stores b if one of its subschemata does. A cover $F \subseteq \mathcal{A}$ *avoids* b if no schema Xa with $X \to a \in F$ stores b. Since $\mathcal{H}_\theta(S) = \sum_b n_S(b)\,\theta(b)$, a schema that dominates S is at least as hot as S under every heat map, and the heat of an avoided attribute adds nothing to the schemata of the cover.

Theorem 26 (Separation criterion)**.** *Let $\mathcal{D}$ be an E-lossless, E-dependency-preserving, E-3NF decomposition whose subschemata have at most N non-key atomic FDs each.*

1) *If every cover contains, for each subschema S of $\mathcal{D}$, a member whose schema dominates S, then $\mathcal{D}$ has heat h^* under every heat map.*
2) *If some cover avoids an attribute that $\mathcal{D}$ stores, then for every L some heat map gives $\mathcal{D}$ a heat above $L \cdot h^*$. Otherwise its heat never exceeds $N \cdot h^*$.*

Indeed, for (1) the cover that attains h^* contains, for each subschema of $\mathcal{D}$, a member whose schema dominates it, so no subschema is hotter than h^* (Lem. 21). For (2), raising the heat of the avoided attribute heats $\mathcal{D}$ but no schema of the avoiding cover, and these schemata bound h^*. Hence frequencies can make a design arbitrarily hotter than the optimum exactly when it stores an attribute that some cover avoids. A design that meets (1) stays optimal however the frequencies drift, so a change of workload never calls for a redesign. On the delivery reduct, $\mathcal{D}_f$ stores the courier, which the cover of $\mathcal{D}_\theta$ avoids, and $\mathcal{D}_\theta$ stores the van, which the cover of $\mathcal{D}_f$ avoids. Sec. VI-D runs both tests on every reduct.

V. Completions and Deferred Maintenance

Unlike the refreshes that Sec. III prices, completions move tuples into scopes and make them subject to the constraints. We price the repair of a conflicting completion, trace conflicts to their sources, and name the design property that decides what they cost.

Definition 27 (Completion mode)**.** A *completion mode* over R is an attribute $A \in E$. An application picks a tuple t with $t(A) = \perp$ that is $(E \setminus \{A\})$-total, and replaces $t(A)$ by some $v \in \mathrm{dom}(A)$, moving t into the scope r^E.

On a decomposed design, applying a completion mode is a *scope entry*: the entering tuple's projections are inserted into every subschema, the tuple is removed from the remainder, and all constraints must then hold. The entry check validates each rule once, while the design decides what repairing a violation costs. The newcomer-wins policy of Ex. 2 lets the entering value prevail over the value that its group holds in the scope.

Proposition 28 (Conflict amplification)**.** *Let $X \to A$ hold on the scope, let the entering tuple t disagree on A with the k scope tuples of its X-group, and let the newcomer-wins policy apply. A design that stores $X \to A$ as a non-key FD on a subschema rewrites there one stored A-value per distinct projection of the group, all k of them whenever the group's tuples have distinct projections on the subschema. A design that stores XA only in subschemata of which X is a key holds the group in one row of each, and rewrites one row in each.*

Hence the redundancy that prices a refresh also prices the repair of a conflict. An earlier refresh can leave such a disagreement behind.

Definition 29 (Latent conflict)**.** Let $X \to A$ hold on the scope r^E. An XA-total tuple $t \in r \setminus r^E$ carries a *latent conflict* with $X \to A$ if a tuple of r^E agrees with t on X but not on A.

No constraint sees a latent conflict, since its carrier lies outside the scope, yet the conflict decides the repair that a completion needs.

Proposition 30 (Dynamics of latent conflicts)**.** *Let the scope of r satisfy $\Sigma[E]$ and let $X \to A$ hold on it.*

1) *A retraction creates no latent conflict.*
2) *A refresh that moves an X-group of the scope to a fresh A-value turns every XA-total pending tuple that agreed with the group into a carrier of a latent conflict, unless the tuple is total on the embedding of the refreshing mode and is refreshed with the group.*
3) *Only a completion turns a latent conflict into a violation, and Prop. 28 gives the cost of its repair.*
4) *An insert into the scope is a scope entry: a disagreement with its X-group is a violation at once, repaired as in Prop. 28. An insert outside the scope violates no constraint of the scope, and carries a latent conflict with $X \to A$ exactly when it is XA-total and disagrees on A with its X-group.*

Indeed, a refresh rewrites only the tuples of the group that are total on the embedding of its mode, so an agreeing pending tuple that is not total on it keeps the old value. Such a tuple lies outside the scope, where no constraint sees it, and only its completion moves it in. Missing values thus postpone maintenance without removing it. By Prop. 30(2), every refresh of a group with agreeing pending tuples that are not total on the mode's embedding makes their later completions conflicting, so the hotter a rule, the more latent conflicts it creates. The workload decides how many arise, and the design decides what each costs (Prop. 28). Completions also cause *completeness drift*: as values are filled, groups grow, and refreshes and repairs cost more.

What a completion costs, at entry and in repair, turns on one property of the design. A mode $(E'; X \to A)$ with $E' \subseteq E$ is *key-materialized* in a design if some subschema contains XA and has X as a key. On such a subschema the rule $X \to A$ is enforced as a certain key [31]: the entry check of a completion is the lookup that its insert performs anyway, and the repair of Prop. 28 rewrites one row there. Otherwise a trigger enforces the rule on a subschema in which its group occupies many rows, and every conflicting completion pays the amplification there. Ex. 31 traces one conflict from its source to its cost.

Example 31 (The life of a latent conflict)**.** The pending tuple t_6 of Table I claims *Ravi* for zone *Z1*, whose scope tuples carry *Maggie*. It acquires that disagreement when *Z1* is reassigned from *Ravi* to *Maggie*, since the mode $(cgz;\ z \to c)$ refreshes no tuple whose network is missing (Prop. 30(2)). Only the completion of the network turns the latent conflict into a violation (Prop. 30(3)). The repair is that of Ex. 2: it rewrites the rows of *Z1* and *Z2* in $bcdz$ on $\mathcal{D}_f$, and on $\mathcal{D}_\theta$, where the mode is key-materialized on cz, one row per zone, found by a key lookup (Prop. 28).

VI. Experiments

A. Research Questions

We structure the evaluation along six questions.

RQ1 How does the cost of maintaining a redundantly stored rule scale with its redundancy, for refreshes and for completions, across designs?

RQ2 How do classical, structure-optimal, and heat-aware synthesis compare in the heat and structure of their designs and in synthesis time?

RQ3 What do workloads that mix refreshes and completions cost as the completion share varies, and does tailoring slow reads?

RQ4 How robust are the designs to completeness drift, as scopes and groups grow, and to misestimated heat?

RQ5 What does a whole declared workload cost, once updates of every kind arrive together over one maintenance window, each at its own rate?

RQ6 What do the updates that the mined FDs govern cost on the design tailored to their mined heat, and as they grow in number?

The six questions build one argument. RQ1 establishes the mechanism, RQ2 measures how often synthesis can exploit it on real reducts, and RQ3 carries it to mixed workloads and to reads. RQ4 tests it under drift and misestimated heat, and RQ5 runs it live over whole windows of opposite skew, as RQ6 does over a real release window.

B. Experimental Setup

Environment. The code, the data, and the experimental protocol are available at https://github.com/zhuoxingzhang/completeness-update-tailored-design. All algorithms are implemented in Python, and the operational studies run on MySQL, on a workstation with a 12-core Intel Core i7-12700 and 128 GB of memory. Unless stated otherwise, a reported time is the median of three repetitions. Every design enforces its minimal keys by unique indexes, and every update rewrites the groups of the non-key FDs it moves.

Datasets. We use 12 real-world incomplete relations standard in data profiling and normalization research [32], [33], of 9 to 38 attributes, 108 to 17,305,799 rows, and 0.2% to 76.2% null cells, with the FD sets mined on them, of 8 to 2,995 FDs. Each mined set is taken as the reduct $\Sigma[E]$ over the full attribute set.

Baselines. We compare three synthesizers that share one front end, so the objective is the only variable. 3NF is the classical lossless, dependency-preserving synthesis [28], [29], SO is the state-of-the-art structure-optimal synthesis [12] run on the reduct, and HA is heat-aware synthesis by Alg. 1.

Heat assignment. Heat is declared in two ways. The graded declaration makes each attribute hot with probability p and draws its heat uniformly from $\{1, \ldots, 10\}$. We sweep p and report means over 20 draws. The worst single attribute gives one attribute heat 10 and every other attribute heat 1, and we report the attribute on which SO and HA differ most.

Controlled studies. Four studies run on generated instances of the delivery schema of Ex. 1 that satisfy the declared rules. The generator fixes the number of zones, so growing the instance grows the group k of deliveries a zone carries. Table II lists the three workloads under which RQ4 and RQ5 issue whole maintenance windows of 1,000 updates. Both studies declare heat by operation, as a log does: every rule that determines the

TABLE II: The three workload profiles (RQ4, RQ5): the updates of one window of 1,000, by kind.

	profile		
update	delivery desk	split	fleet office
refresh the courier of a zone	300	160	20
refresh the van of a courier	20	160	300
refresh the zone of a delivery	50	50	50
refresh the round of a van	30	30	30
refresh the branch of a round	10	10	10
refresh the network of a van	10	10	10
complete a delivery, clean	120	120	120
complete a delivery, conflicting	20	20	20
retract a value	50	50	50
insert an E-total delivery	200	200	200
insert a pending delivery	100	100	100
delete a delivery	90	90	90

attribute an operation rewrites carries the rate of that operation (Lem. 6). A conflicting completion enters a tuple that disagrees with its group on the attribute the window rewrites most often (Prop. 30(2)), so its rate is added to the heat of that attribute's rules, the courier's at the delivery desk and the split profile and the van's at the fleet office.

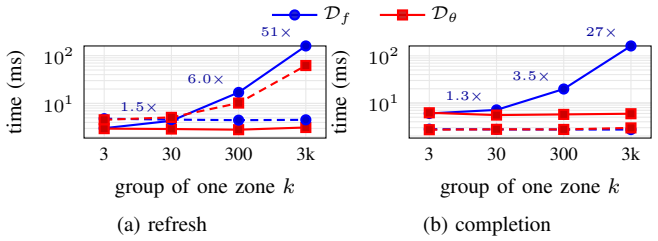
Real release. RQ6 uses 23 columns of the COVID table of Our World in Data, which is revised in place in a repository that keeps every edition.[1] The design scope E holds 21 of them, leaving a third of the tuples E-total. A window is a pair of editions, and its workload is the difference between the two, coalesced into the updates that produce it. We use two consecutive windows, which share the *base edition*, the relation that the designs store. The *mining window* ends in the base edition: the constraints are mined on its earlier edition, and the heat of an attribute A is the number of its updates that refresh A inside the scope (Lem. 6), while an attribute the mining window does not refresh keeps the default heat 1. The *release window* starts from the base edition, and every reported time is measured on its updates, none of which takes part in the mining. An update is priced if some FD of the reduct determines the attribute it refreshes. We report the mean total time of the priced updates.

Re-numbered grades. To test the designs as the priced updates grow in number, RQ6 grades the priced attributes and re-numbers the grades. A priced attribute with n updates in the scope receives one of ten grades ℓ in proportion to n. The maps ℓ^2, 2^ℓ, and ℓ^3 re-number these grades, and the updates of each attribute are multiplied to follow its new grade.

C. Operational Costs under Controlled Redundancy (RQ1)

RQ1 measures the case study of Exs. 1 and 2 on instances that grow. Fig. 2 shows that redundancy governs the cost of maintaining a rule stored as a non-key FD. One reassignment rewrites every occurrence a zone carries on $\mathcal{D}_f$ and a constant number of rows on $\mathcal{D}_\theta$, the group rewrite of Sec. III-B. The time follows that count and parts the designs by $51\times$ at the largest group (Fig. 2(a)). A van swap is the same mechanism with the designs exchanged, so the mix of refreshes decides

[1] ourworldindata.org/coronavirus, github.com/owid/covid-19-data



(a) refresh (b) completion

Fig. 2: Scaling the group k of one zone on the delivery designs (RQ1): (a) one reassignment (solid) and one van swap (dashed), (b) one entering tuple, conflicting (solid) and clean (dashed); labels give the time on $\mathcal{D}_f$ over that on $\mathcal{D}_\theta$.

TABLE III: Design heat under the graded declaration at $p = 0.5$ (RQ2), totals rounded, HA bold where strictly coolest; *: SO and HA have equal heat in every draw at every p.

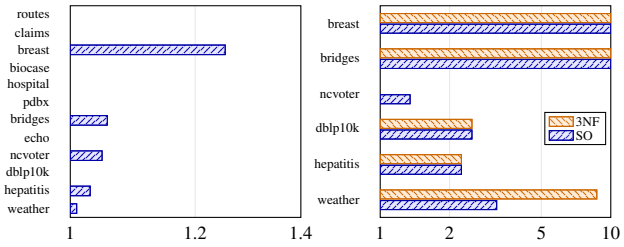
		maximal			total					maximal			total		
reduct	$\lvert\mathcal{A}\rvert$	3NF	SO	HA	3NF	SO	HA	reduct	$\lvert\mathcal{A}\rvert$	3NF	SO	HA	3NF	SO	HA
routes*	24	2.6	2.6	2.6	3	3	3	bridges	128	7.0	6.7	**6.3**	36	26	**24**
claims*	28	0.0	0.0	0.0	0	0	0	echo*	355	11.1	9.9	9.9	41	31	31
breast	47	4.2	4.2	**3.4**	6	6	**5**	ncvoter	1,249	22.8	16.6	**15.8**	160	85	**77**
biocase*	47	0.0	0.0	0.0	0	0	0	dblp10k	1,377	23.6	23.3	23.3	363	345	**342**
hospital*	83	8.0	0.0	0.0	11	0	0	hepatitis	4,513	57.8	57.5	**55.9**	2,393	2,203	**2,110**
pdbx*	101	4.8	4.8	4.8	17	17	17	weather	6,582	54.6	43.0	**42.6**	4,108	3,374	**3,271**

which design is cheaper. Completions come in the two kinds of scope entry (Fig. 2(b)). Clean completions write the same rows and take the same time on both designs at every scale, since both issue the same inserts. Conflicting completions repair a latent conflict, which rewrites the same rows as a reassignment and takes up to $27\times$ as long on $\mathcal{D}_f$.

D. Schema-Level Gains on Real Reducts (RQ2)

RQ1 shows that a design sets the cost of a hot rule by whether it stores the rule under a key. RQ2 asks how often synthesis has that choice on the reducts of real relations, and what taking it costs in structure and in time. The two declarations test different things: the graded declaration asks whether the objective ever loses to structure, and the worst single attribute, the shape of the workload of RQ1, asks how far the objectives diverge.

Graded declaration. Table III shows that heat-aware synthesis is never hotter than either baseline on the maximum, as Thm. 23 guarantees. On these reducts, it is never hotter on the mean total heat over all subschemata either, at every p of the sweep. In fact, on the eight smallest reducts, an exhaustive search finds no elimination order that lowers the total in any draw. On six reducts, those marked in Table III, the structure-optimal design meets the condition of Thm. 26(1), so no declaration can separate it from the heat-aware one, and no drift of the frequencies calls for a redesign. On the other six, where synthesis has a choice, the maximum and the total of heat-aware synthesis fall below the structure-optimal ones on 4 to 6, depending on p. Against classical synthesis the gain can be the whole retained heat, as on hospital, where structural optimization alone removes it and heat-aware



(a) graded declaration ($p = 0.5$) (b) worst single attribute

Fig. 3: Maximal heat of the baselines as a multiple of HA's on the 12 reducts (RQ2): (a) SO under the graded declaration; (b) 3NF and SO under the worst single attribute, the reducts without a separating attribute omitted. A missing bar is a tie.

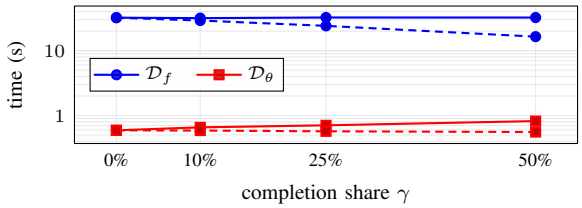


Fig. 4: Mixed workloads at a group of 3,000 (RQ3): time for 200 updates as the completion share γ grows, with conflicting (solid) and clean (dashed) completions.

synthesis inherits the removal by Thm. 23. The objective costs little structure and little time. Heat-aware synthesis attains E-BCNF wherever the structure-optimal design does, and at $p = 0.5$ it needs fewer subschemata than that design on seven of the twelve reducts. Synthesis takes at most 8.2 s, and wherever it takes longer than 10 ms, heat-aware synthesis takes at most $1.07\times$ the time of structure-optimal synthesis.

Worst single attribute. Fig. 3 shows that the objectives diverge most where the heat concentrates on one attribute. Spread over many attributes by the graded declaration, heat makes SO hotter than HA on five of the twelve reducts (Fig. 3(a)). The worst single attribute separates six of them, by up to $10\times$ (Fig. 3(b)). There SO stores rules of the hot attribute as non-key FDs, while HA stores fewer of them, and none on four of the six. On five of the six, some cover avoids an attribute that SO stores, so a hotter declaration of that attribute separates the designs without bound (Thm. 26(2)). On ncvoter the hot attribute even drives SO above 3NF, the converse of hospital, where 3NF retains the heat that SO removes. Hence neither frequency-blind synthesizer dominates, and the number of non-key FDs is the wrong measure where the workload concentrates on one attribute.

E. Mixed Refresh–Completion Workloads (RQ3)

RQ1 times each operation in isolation. RQ3 interleaves refreshes with completions, and then turns to reads. Fig. 4 shows that the advantage on refreshes survives completions. The workload of pure refreshes separates the designs by $55\times$.

TABLE IV: Reads on the delivery designs at 10^6 tuples (RQ3).

		$\mathcal{D}_f$	$\mathcal{D}_\theta$	$\mathcal{D}_f/\mathcal{D}_\theta$
scope reconstruction	ms	216	220	$0.98\times$
delivery by key	ms	0.13	0.13	$1.00\times$
history of one courier	ms	43.7	38.3	$1.14\times$

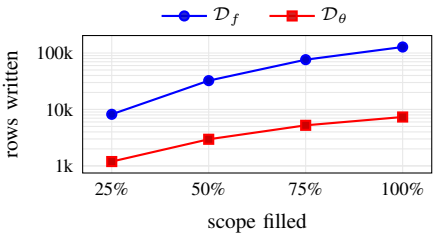


Fig. 5: Rows that one window of 1,000 updates writes at the delivery desk as the scope fills, up to a full group of 192 (RQ4).

Clean completions cost the same on both designs, so as the completion share γ grows the time on $\mathcal{D}_f$ falls while that on $\mathcal{D}_\theta$ stays flat, and the ratio stays above $29\times$. Conflicting completions instead repair a latent conflict, which costs about what one reassignment costs, so the ratio declines only to $40\times$ and the absolute gap stays where it is.

Table IV shows that tailoring does not slow reads. Reconstructing the scope and finding one delivery by its key take about the same time on both designs at 10^6 tuples, since each joins six subschemata of which exactly one is large. The history of one courier runs $1.14\times$ faster on $\mathcal{D}_\theta$, which reads the zones of a courier off the twelve-row subschema cz before it touches the large one.

F. Robustness under Drift and Misestimation (RQ4)

RQ1 and RQ3 concentrate the workload on one hot operation, on a fixed scope and under an exact declaration. RQ4 issues the whole windows of Table II, lets the scope fill, and lets the declaration name the wrong operation. Fig. 5 shows that completeness drift widens the separation: as the scope fills from a quarter to full at the delivery desk, what the window writes grows on both designs, and the ratio rises from $6.9\times$ to $17.4\times$. Indeed, completeness drift is scope entry (Def. 27) in bulk: every van whose network is registered grows a zone, and with it the redundancy of every rule over it. A design chosen today thus gains value as the data completes, provided the declaration names the right operation.

Table V shows that only a declaration by the measured rates lets synthesis return the cheaper design under all three profiles. Declaring uniformly, which counts non-key FDs (Prop. 15), returns $\mathcal{D}_f$ everywhere and pays up to $17.5\times$ at the delivery desk, and even at the fleet office it misses the cheaper variant of Ex. 24 that the rates select. Declaring by group size returns $\mathcal{D}_\theta$ everywhere and pays $3.4\times$ at the fleet office. Indeed, a reassignment rewrites larger groups than a van swap, yet at the fleet office van swaps arrive fifteen times as often.

TABLE V: What a declaration costs (RQ4): rows that one window of 1,000 updates writes on the heat-aware design, as a multiple of the cheaper design's, at a group of 192. A profile is served right where the factor is 1.00.

heat declared	delivery desk	split	fleet office
by the measured rates	1.00×	1.00×	1.00×
uniformly	17.53×	3.71×	1.28×
by the group each rule ranges over	1.00×	1.00×	3.38×
with the two rates exchanged	17.46×	3.63×	3.38×

TABLE VI: One maintenance window of 1,000 updates over 36,864 tuples in groups of 3,072 (RQ5): rows written and update time on each design under the three rate profiles.

	rows written			update time (s)		
window	$\mathcal{D}_f$	$\mathcal{D}_\theta$	ratio	$\mathcal{D}_f$	$\mathcal{D}_\theta$	ratio
delivery desk	1,980,617	41,907	47.26×	54.14	3.02	17.90×
split	1,121,121	258,834	4.33×	32.26	6.39	5.05×
fleet office	138,639	537,797	0.26×	6.46	10.92	0.59×

G. A Window at Declared Rates (RQ5)

RQ4 counts the rows a window writes. RQ5 times the three workloads of Table II live, every kind of update the schema admits in mixed order and at its own rate.

The three profiles issue 15, 1, and 1/15 reassignments per van swap, and the mix of the two refreshes decides which design is cheaper (RQ1). Of the three synthesizers, only the heat-aware one follows the rates. Over 200 orders of the atomic closure, structure-optimal synthesis returns $\mathcal{D}_f$ or its variant of Ex. 24, the $\mathcal{D}_f$ family, every time, since $bdvz$ carries two non-key FDs against the one of $bcdz$. Classical synthesis, which ranks nothing, returns that family or the $\mathcal{D}_\theta$ family as the input order decides, and heat-aware synthesis returns the family the rates point to in all 200 orders, at the delivery desk and at the fleet office alike.

Table VI shows that following the profile pays in time as it does in rows. At the delivery desk $\mathcal{D}_\theta$ finishes in 3.02 s where $\mathcal{D}_f$ takes 54.14 s. At the fleet office the same two designs change places, because the traffic now swaps vans, and $\mathcal{D}_\theta$ stores the van under the non-key FD $bz \to v$ of $bdvz$ (Ex. 10).

Fig. 6 shows that the advantage of following the rates grows with the groups, at both profiles. As the groups grow sixteenfold, the design each profile favors takes at most 2.2× as long, while the other takes 3.3 to 10.4× as long. This is the group rewrite of Sec. III-B in the time domain.

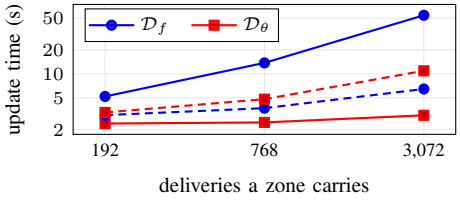


Fig. 6: The same window as the group of a zone grows (RQ5): the desk profile (solid), which reassigns couriers, and the fleet profile (dashed), which swaps vans.

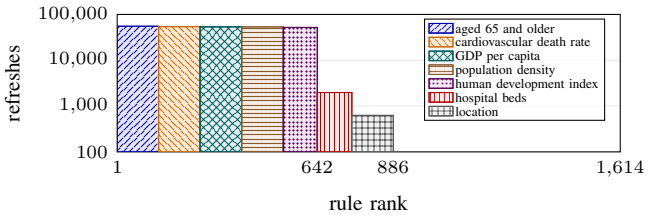


Fig. 7: Refreshes per rule in the mining window (RQ6): rules ranked by refreshes, colored by the attribute they determine.

TABLE VII: The base edition and the release window (RQ6) and, per design, the subschemata, the maximal heat, and the time of the priced updates under the observed and re-numbered grades; bold marks the least per line.

base edition and release window			
tuples	168,655	tuples in the scope	58,802
atomic FDs of the reduct	1,614	minimal keys	1
values refreshed in the 7 priced attributes	270,924	coalesced priced updates	409

the designs	*3NF*	*SO*	*HA*
subschemata	130	137	**128**
maximal heat (updates)	79	11	**1**
refresh time (s) of the priced updates	*3NF*	*SO*	*HA*
ℓ, 409 priced updates	111.3	55.0	**25.7**
ℓ^2, 3,815 priced updates	1,258.6	648.5	**255.3**
2^ℓ, 13,575 priced updates	4,218.0	2,415.4	**1,201.7**
ℓ^3, 34,469 priced updates	11,644.8	6,707.7	**3,404.9**

H. A Real Release Window (RQ6)

RQ5 shows that the declared rates decide which design is cheaper. RQ6 mines the rates before a real release. The mining window runs from the edition of 31 March 2024 to the base edition of 30 June 2024. The release window runs from the base edition to the edition of 19 August 2024, between which country-level attributes are corrected.

Fig. 7 shows that the refreshed values of the priced attributes are skewed across the rules of the reduct. The rules of five country-level attributes receive 99% of them, and 728 of 1,614 rules receive none. Every rule that determines an attribute receives all of its refreshes, since a refresh moves a group of every such rule (Lem. 6). Hence a design avoids the heat of an attribute only by storing none of its rules as a non-key FD.

Table VII shows that the coolest design, HA, runs the priced updates fastest in every repetition, by 2.14× against SO and 4.33× against 3NF. Fig. 8 shows that the gap persists as the priced updates grow in number: across the observed and the re-numbered grades, SO takes 1.97 to 2.54× the time of HA, and 3NF 3.42 to 4.93× (Table VII).

The times rank the designs as their heat does (Table VII). Indeed, 3NF stores three rules deriving the human development index as non-key FDs in four subschemata, and each priced update of the index refreshes all four. SO stores one priced rule as a non-key FD, and HA stores none: its non-key FDs determine only attributes of the default heat 1. Over

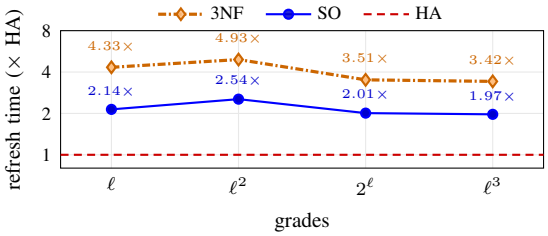


Fig. 8: Refresh time of the priced updates on 3NF and SO as a multiple of HA's (RQ6), under the observed grades ℓ and their re-numberings: the dashed line marks HA.

the whole window, which adds 253,041 updates that are not priced, SO and 3NF still take $1.67\times$ and $2.38\times$ as long as HA.

Summary. Redundancy amplifies the cost of every refresh and every conflicting completion of a rule stored as a non-key FD, so a design sets that cost when it chooses which rules to store under a key (RQ1). Heat-aware synthesis makes that choice on real reducts at little cost in structure and time: it is never hotter than either baseline, and up to $10\times$ cooler than the structure-optimal design (RQ2). The gain survives workloads that mix refreshes with completions and does not slow reads (RQ3). It grows as the scope fills, and only a declaration by the measured rates secures it under every profile (RQ4). Over a whole window, the design that follows the rates finishes up to $17.9\times$ faster on the delivery schema (RQ5), and on a real release the coolest design runs the priced updates fastest (RQ6).

VII. Related Work

Constraints and design under incompleteness. FDs over the incomplete relations of [20], [21], [34] were axiomatized under no-information [35] and weak/strong semantics [36], [37] and surveyed in [38]. Implication under NOT NULL [39], SQL normal forms [24], [40], keys under missing values [31], [41], and cardinality constraints with FDs [42] followed. The design theory of Levene and Loizou [43], which applies its FDs to every tuple regardless of missing values, or-set expansion into possible worlds [44], and certainty-graded FDs [45] take neither completeness requirements nor update frequencies as input. Embedded FDs and UCs, their discovery, and completeness-tailored normalization [16], [17], [46], [47] take completeness as input and underlie this paper.

Updates and queries over incomplete data. Updates of incomplete databases have been studied model-theoretically [48]–[50], whereas our taxonomy is operational. Certain answers over SQL nulls [15], [51], [52] and query completeness [53] concern queries, and the latter marks complete parts of the data, as scopes do.

Normal forms and their justification. Synthesis and its guarantees originate in [19], [28], [29], and its target normal form was refined in [54]. Normal forms have been justified semantically by redundancy and anomalies [18], [22], [25], [26], [55], [56]. Structural parameterizations of BCNF and 3NF [12], [57] and mixed covers [1], [58] optimize update-relevant structure over complete relations, counting the non-key FDs and the minimal keys. Cardinality-bounded design [13], [14] bounds how many tuples share a value of the left-hand side of a non-trivial FD. Neither line of work asks how often a rule is updated.

Integrity maintenance and repair. Simplified integrity checking [59] and production rules derived from constraints [60] enforce a declared constraint on every write, but take the schema, which decides how many rows each write touches, as given. Repairing violations by value modification is NP-hard already for FDs [61], [62], and repairs can weigh several constraints against one another [63]. In the repair of Prop. 28, the policy of the application fixes which values change, and the design sets how many stored rows that takes.

Workload-driven design at other layers. Access frequencies entered design four decades ago, in affinity-based vertical partitioning [6], and later in physical design advisors [7]–[9], hot-cold classification [2]–[4], and workload-driven partitioning [64], [65]. Vertical partitioning decomposes by attribute affinity, not by constraints, so it neither guarantees dependency preservation nor targets a normal form. The others, like materialized views and partial indexes for hot rows [66], tune beneath a fixed schema and inherit the redundancy it admits. Schema design for NoSQL stores [67], [68] and relational denormalization [69] also start from a workload, but add redundancy for reads and preserve no constraints. No prior work tailors logical design to the completeness requirements and the update frequencies of an application at once.

VIII. Conclusion and Future Work

We have introduced completeness- and update-tailored database design, in which the application declares the completeness requirement under which each business rule applies and how often each attribute is updated. Bounding the heat of a schema yields the family of (h, E)-tailored normal forms between E-BCNF and E-3NF. We have characterized the minimum heat of the lossless, dependency-preserving E-3NF designs over the scope, given a synthesis algorithm that attains it, and shown when frequencies can separate a design from it. Our experiments have shown that the update frequencies, and not the structure, decide which design is cheaper, and that choosing by structure can cost more than an order of magnitude. On twelve real-world incomplete relations, heat-aware synthesis is never hotter than the baselines and cools the hottest subschema on about half of them, and on a real release its design runs the priced updates fastest.

Future work will combine heat with cardinality-bounded design [14], which bounds the group sizes that multiply each heat, and ask which designs attain both bounds at once.

AI-Generated Content Acknowledgement

The authors used the AI assistant Claude to polish the language of all sections and to write part of the code behind the experiments of Sec. VI. The authors reviewed all AI-assisted

text and code and take full responsibility for the content of this paper.

REFERENCES

[1] Z. Zhang and S. Link, "Mixed covers of keys and functional dependencies for maintaining the integrity of data under updates," *Proc. VLDB Endow.*, vol. 17, no. 7, pp. 1578–1590, 2024.

[2] T. Härder, "Handling hot spot data in db-sharing systems," *Information Systems*, vol. 13, no. 2, pp. 155–166, 1988.

[3] J. J. Levandoski, P. Larson, and R. Stoica, "Identifying hot and cold data in main-memory databases," in *29th IEEE International Conference on Data Engineering (ICDE), Brisbane, Australia*, 2013, pp. 26–37.

[4] J. A. DeBrabant, A. Pavlo, S. Tu, M. Stonebraker, and S. B. Zdonik, "Anti-caching: A new approach to database management system architecture," *Proc. VLDB Endow.*, vol. 6, no. 14, pp. 1942–1953, 2013.

[5] Transaction Processing Performance Council, "TPC benchmark C standard specification, revision 5.11," https://www.tpc.org/tpcc/, 2010.

[6] S. B. Navathe, S. Ceri, G. Wiederhold, and J. Dou, "Vertical partitioning algorithms for database design," *ACM Trans. Database Syst.*, vol. 9, no. 4, pp. 680–710, 1984.

[7] S. Chaudhuri and V. R. Narasayya, "Self-tuning database systems: A decade of progress," in *VLDB*. ACM, 2007, pp. 3–14.

[8] S. Agrawal, S. Chaudhuri, L. Kollár, A. P. Marathe, V. R. Narasayya, and M. Syamala, "Database tuning advisor for Microsoft SQL server 2005," in *VLDB*. Morgan Kaufmann, 2004, pp. 1110–1121.

[9] D. C. Zilio, J. Rao, S. Lightstone, G. M. Lohman, A. J. Storm, C. Garcia-Arellano, and S. Fadden, "DB2 design advisor: Integrated automatic physical database design," in *VLDB*. Morgan Kaufmann, 2004, pp. 1087–1097.

[10] J. C. Westland, "Economic incentives for database normalization," *Inf. Process. Manag.*, vol. 28, no. 5, pp. 647–662, 1992.

[11] H. Lee, "Justifying database normalization: A cost/benefit model," *Inf. Process. Manag.*, vol. 31, no. 1, pp. 59–67, 1995.

[12] Z. Zhang and S. Link, "Synthesizing third normal form schemata that minimize integrity maintenance and update overheads: Parameterizing 3nf by the numbers of minimal keys and functional dependencies," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 225:1–225:25, 2025.

[13] S. Link and Z. Wei, "Logical schema design that quantifies update inefficiency and join efficiency," in *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, 2021, pp. 1169–1181.

[14] Z. Wei and S. Link, "The bounded cardinality normal form for the logical design of relational database schemata," *ACM Trans. Database Syst.*, vol. 50, no. 4, pp. 15:1–15:45, 2025.

[15] L. Libkin, "Incomplete data: what went wrong, and how to fix it," in *Proceedings of the 33rd ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems, PODS'14, Snowbird, UT, USA, June 22-27, 2014*, R. Hull and M. Grohe, Eds. ACM, 2014, pp. 1–13. [Online]. Available: https://doi.org/10.1145/2594538.2594561

[16] Z. Wei and S. Link, "Embedded functional dependencies and data-completeness tailored database design," *Proc. VLDB Endow.*, vol. 12, no. 11, pp. 1458–1470, 2019.

[17] Z. Wei and S. Link, "Embedded functional dependencies and data-completeness tailored database design," *ACM Trans. Database Syst.*, vol. 46, no. 2, pp. 7:1–7:46, 2021.

[18] M. W. Vincent, "Semantic foundations of 4nf in relational database design," *Acta Informatica*, vol. 36, no. 3, pp. 173–213, 1999.

[19] C. Beeri and P. A. Bernstein, "Computational problems related to the design of normal form relational schemas," *ACM Trans. Database Syst.*, vol. 4, no. 1, pp. 30–59, 1979.

[20] Y. E. Lien, "On the equivalence of database models," *J. ACM*, vol. 29, no. 2, pp. 333–362, 1982. [Online]. Available: https://doi.org/10.1145/322307.322311

[21] C. Zaniolo, "Database relations with null values," *J. Comput. Syst. Sci.*, vol. 28, no. 1, pp. 142–166, 1984. [Online]. Available: https://doi.org/10.1016/0022-0000(84)90080-1

[22] S. Kolahi and L. Libkin, "On redundancy vs dependency preservation in normalization: an information-theoretic study of 3nf," in *Proceedings of the Twenty-Fifth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS), Chicago, Illinois, USA*, 2006, pp. 114–123.

[23] H. Köhler, "Finding faithful boyce-codd normal form decompositions," in *Algorithmic Aspects in Information and Management, Second International Conference, AAIM 2006, Hong Kong, China, June 20-22, 2006, Proceedings*, 2006, pp. 102–113.

[24] H. Köhler and S. Link, "SQL schema design: Foundations, normal forms, and normalization," in *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016, San Francisco, CA, USA, June 26 - July 01, 2016*, 2016, pp. 267–279.

[25] M. W. Vincent, "Redundancy elimination and a new normal form for relational database design," in *Semantics in Databases, Selected Papers from a Workshop, Prague, Czech Republic, 1995*, ser. Lecture Notes in Computer Science, L. Libkin and B. Thalheim, Eds., vol. 1358. Springer, 1995, pp. 247–264. [Online]. Available: https://doi.org/10.1007/BFb0035011

[26] M. Arenas and L. Libkin, "An information-theoretic approach to normal forms for relational and XML data," *J. ACM*, vol. 52, no. 2, pp. 246–283, 2005.

[27] J. H. Jou and P. C. Fischer, "The complexity of recognizing 3nf relation schemes," *Inf. Process. Lett.*, vol. 14, no. 4, pp. 187–190, 1982.

[28] P. A. Bernstein, "Synthesizing third normal form relations from functional dependencies," *ACM Trans. Database Syst.*, vol. 1, no. 4, pp. 277–298, 1976.

[29] J. Biskup, U. Dayal, and P. A. Bernstein, "Synthesizing independent database schemas," in *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data, Boston, Massachusetts, USA, May 30 - June 1*, 1979, pp. 143–151.

[30] C. L. Lucchesi and S. L. Osborn, "Candidate keys for relations," *J. Comput. Syst. Sci.*, vol. 17, no. 2, pp. 270–279, 1978.

[31] H. Köhler, U. Leck, S. Link, and X. Zhou, "Possible and certain keys for SQL," *VLDB J.*, vol. 25, no. 4, pp. 571–596, 2016. [Online]. Available: https://doi.org/10.1007/s00778-016-0430-9

[32] T. Papenbrock and F. Naumann, "A hybrid approach to functional dependency discovery," in *SIGMOD*, 2016, pp. 821–833.

[33] T. Papenbrock and F. Naumann, "Data-driven schema normalization," in *Proceedings of the 20th International Conference on Extending Database Technology, EDBT 2017, Venice, Italy, March 21-24, 2017*, 2017, pp. 342–353.

[34] E. F. Codd, "Extending the database relational model to capture more meaning," *ACM Trans. Database Syst.*, vol. 4, no. 4, pp. 397–434, 1979. [Online]. Available: https://doi.org/10.1145/320107.320109

[35] P. Atzeni and N. M. Morfuni, "Functional dependencies and constraints on null values in database relations," *Inf. Control.*, vol. 70, no. 1, pp. 1–31, 1986. [Online]. Available: https://doi.org/10.1016/S0019-9958(86)80022-5

[36] M. Levene and G. Loizou, "Axiomatisation of functional dependencies in incomplete relations," *Theor. Comput. Sci.*, vol. 206, no. 1-2, pp. 283–300, 1998. [Online]. Available: https://doi.org/10.1016/S0304-3975(98)80029-7

[37] M. Levene and G. Loizou, *A guided tour of relational databases and beyond*. Springer, 1999.

[38] A. Badia and D. Lemire, "Functional dependencies with null markers," *Comput. J.*, vol. 58, no. 5, pp. 1160–1168, 2015. [Online]. Available: https://doi.org/10.1093/comjnl/bxu039

[39] S. Hartmann and S. Link, "The implication problem of data dependencies over SQL table definitions: Axiomatic, algorithmic and logical characterizations," *ACM Trans. Database Syst.*, vol. 37, no. 2, pp. 13:1–13:40, 2012. [Online]. Available: https://doi.org/10.1145/2188349.2188355

[40] H. Köhler and S. Link, "SQL schema design: foundations, normal forms, and normalization," *Inf. Syst.*, vol. 76, pp. 88–113, 2018.

[41] H. Köhler, S. Link, and X. Zhou, "Possible and certain SQL key," *Proc. VLDB Endow.*, vol. 8, no. 11, pp. 1118–1129, 2015. [Online]. Available: http://www.vldb.org/pvldb/vol8/p1118-koehler.pdf

[42] S. Link, H. Koehler, A. Gandhi, S. Hartmann, and B. Thalheim, "Cardinality constraints and functional dependencies in SQL: taming data redundancy in logical database design," *Inf. Syst.*, vol. 115, p. 102208, 2023. [Online]. Available: https://doi.org/10.1016/j.is.2023.102208

[43] M. Levene and G. Loizou, "Database design for incomplete relations," *ACM Trans. Database Syst.*, vol. 24, no. 1, pp. 80–125, 1999. [Online]. Available: https://doi.org/10.1145/310701.310712

[44] L. Libkin, "Normalizing incomplete databases," in *Proceedings of the Fourteenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, May 22-25, 1995, San Jose, California, USA*,

M. Yannakakis and S. Abiteboul, Eds. ACM Press, 1995, pp. 219–230. [Online]. Available: https://doi.org/10.1145/212433.220217

[45] S. Link and H. Prade, "Relational database schema design for uncertain data," *Inf. Syst.*, vol. 84, pp. 88–110, 2019.

[46] Z. Wei, U. Leck, and S. Link, "Discovery and ranking of embedded uniqueness constraints," *Proc. VLDB Endow.*, vol. 12, no. 13, pp. 2339–2352, 2019. [Online]. Available: http://www.vldb.org/pvldb/vol12/p2339-wei.pdf

[47] Z. Wei, S. Hartmann, and S. Link, "Algorithms for the discovery of embedded functional dependencies," *VLDB J.*, vol. 30, no. 6, pp. 1069–1093, 2021. [Online]. Available: https://doi.org/10.1007/s00778-021-00684-3

[48] R. Fagin, J. D. Ullman, and M. Y. Vardi, "On the semantics of updates in databases," in *Proceedings of the Second ACM SIGACT-SIGMOD Symposium on Principles of Database Systems, March 21-23, 1983, Colony Square Hotel, Atlanta, Georgia, USA*, R. Fagin and P. A. Bernstein, Eds. ACM, 1983, pp. 352–365. [Online]. Available: https://doi.org/10.1145/588058.588100

[49] S. Abiteboul and G. Grahne, "Update semantics for incomplete databases," in *VLDB'85, Proceedings of 11th International Conference on Very Large Data Bases, August 21-23, 1985, Stockholm, Sweden*, A. Pirotte and Y. Vassiliou, Eds. Morgan Kaufmann, 1985, pp. 1–12. [Online]. Available: http://www.vldb.org/conf/1985/P001.PDF

[50] M. Winslett, "A model-based approach to updating databases with incomplete information," *ACM Trans. Database Syst.*, vol. 13, no. 2, pp. 167–196, 1988. [Online]. Available: https://doi.org/10.1145/42338.42386

[51] L. Libkin, "Sql's three-valued logic and certain answers," *ACM Trans. Database Syst.*, vol. 41, no. 1, pp. 1:1–1:28, 2016. [Online]. Available: https://doi.org/10.1145/2877206

[52] P. Guagliardo and L. Libkin, "On the codd semantics of SQL nulls," *Inf. Syst.*, vol. 86, pp. 46–60, 2019. [Online]. Available: https://doi.org/10.1016/j.is.2018.08.001

[53] S. Razniewski and W. Nutt, "Completeness of queries over incomplete databases," *Proc. VLDB Endow.*, vol. 4, no. 11, pp. 749–760, 2011. [Online]. Available: http://www.vldb.org/pvldb/vol4/p749-razniewski.pdf

[54] T. W. Ling, F. W. Tompa, and T. Kameda, "An improved third normal form for relational databases," *ACM Trans. Database Syst.*, vol. 6, no. 2, pp. 329–346, 1981.

[55] S. Kolahi and L. Libkin, "An information-theoretic analysis of worst-case redundancy in database design," *ACM Trans. Database Syst.*, vol. 35, no. 1, pp. 5:1–5:32, 2010. [Online]. Available: https://doi.org/10.1145/1670243.1670248

[56] S. Kolahi, "Dependency-preserving normalization of relational and XML data," *J. Comput. Syst. Sci.*, vol. 73, no. 4, pp. 636–647, 2007.

[57] Z. Zhang, W. Chen, and S. Link, "Composite object normal forms: Parameterizing boyce-codd normal form by the number of minimal keys," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 13:1–13:25, 2023.

[58] Z. Zhang and S. Link, "Mixed covers: optimizing updates and queries using minimal keys and functional dependencies," *The VLDB Journal*, vol. 34, no. 3, 2025.

[59] J. Nicolas, "Logic for improving integrity checking in relational data bases," *Acta Informatica*, vol. 18, pp. 227–253, 1982. [Online]. Available: https://doi.org/10.1007/BF00263192

[60] S. Ceri and J. Widom, "Deriving production rules for constraint maintainance," in *16th International Conference on Very Large Data Bases, August 13-16, 1990, Brisbane, Queensland, Australia, Proceedings*, D. McLeod, R. Sacks-Davis, and H. Schek, Eds. Morgan Kaufmann, 1990, pp. 566–577. [Online]. Available: http://www.vldb.org/conf/1990/P566.PDF

[61] P. Bohannon, W. Fan, M. Flaster, and R. Rastogi, "A cost-based model and effective heuristic for repairing constraints by value modification," in *Proceedings of the ACM SIGMOD International Conference on Management of Data, Baltimore, Maryland, USA, June 14-16, 2005*, F. Özcan, Ed. ACM, 2005, pp. 143–154. [Online]. Available: https://doi.org/10.1145/1066157.1066175

[62] S. Kolahi and L. V. S. Lakshmanan, "On approximating optimum repairs for functional dependency violations," in *Database Theory - ICDT 2009, 12th International Conference, St. Petersburg, Russia, March 23-25, 2009, Proceedings*, ser. ACM International Conference Proceeding Series, R. Fagin, Ed., vol. 361. ACM, 2009, pp. 53–62. [Online]. Available: https://doi.org/10.1145/1514894.1514901

[63] X. Chu, I. F. Ilyas, and P. Papotti, "Holistic data cleaning: Putting violations into context," in *29th IEEE International Conference on Data Engineering, ICDE 2013, Brisbane, Australia, April 8-12, 2013*, C. S. Jensen, C. M. Jermaine, and X. Zhou, Eds. IEEE Computer Society, 2013, pp. 458–469. [Online]. Available: https://doi.org/10.1109/ICDE.2013.6544847

[64] C. Curino, E. P. C. Jones, Y. Zhang, and S. Madden, "Schism: a workload-driven approach to database replication and partitioning," *Proc. VLDB Endow.*, vol. 3, no. 1, pp. 48–57, 2010.

[65] A. Pavlo, C. Curino, and S. B. Zdonik, "Skew-aware automatic database partitioning in shared-nothing, parallel OLTP systems," in *Proceedings of the ACM SIGMOD International Conference on Management of Data, SIGMOD 2012, Scottsdale, AZ, USA, May 20-24, 2012*, 2012, pp. 61–72.

[66] M. Stonebraker, "The case for partial indexes," *SIGMOD Rec.*, vol. 18, no. 4, pp. 4–11, 1989.

[67] M. J. Mior, K. Salem, A. Aboulnaga, and R. Liu, "Nose: Schema design for nosql applications," in *32nd IEEE International Conference on Data Engineering (ICDE), Helsinki, Finland*, 2016, pp. 181–192.

[68] M. J. Mior, K. Salem, A. Aboulnaga, and R. Liu, "Nose: Schema design for nosql applications," *IEEE Trans. Knowl. Data Eng.*, vol. 29, no. 10, pp. 2275–2289, 2017.

[69] G. L. Sanders and S. Shin, "Denormalization effects on performance of RDBMS," in *34th Annual Hawaii International Conference on System Sciences (HICSS-34), Maui, Hawaii, USA*. IEEE Computer Society, 2001.

[70] D. Maier, *The Theory of Relational Databases*. Computer Science Press, 1983.

APPENDIX

The proofs follow the order in which the results appear in the text.

Proof of Lem. 6. Let $X \to a$ be a non-trivial FD implied by $\Sigma[E]$. Since $a \notin X$, the rewriting leaves every X-value unchanged, so the X-groups of r' are those of r^E. If r' satisfies $X \to a$ and $t \in T$, then every tuple of the X-group of t carries v in r', and belongs to T since v is fresh. Hence T is closed under the X-groups of every such FD, that is, T is a union of components. Conversely, if T is a union of components, every such X-group lies within T or outside it, so in r' it carries v throughout or is unchanged, and $X \to a$ holds as it holds in r^E. □

Proof of Lem. 8. Let $E_i \subseteq E$ be the embedding of μ_i, so every E-total tuple is E_i-total and $r^E \subseteq r^{E_i}$. An application of μ_i refreshes the A-value of the tuples of r^{E_i} that agree with its condition on the determinant of μ_i, hence, restricted to the stored scope, of a group of r^E, which is non-empty when the application refreshes a tuple of the scope. Keeping $\Sigma[E]$ satisfied rewrites the component of A that contains this group (Lem. 6). That component is a union of X-groups for every non-trivial FD $X \to A$ that $\Sigma[E]$ implies, and contains at least one of them, since every tuple of the scope is X-total. Every application of every μ_i that refreshes the scope thus refreshes a group of every such FD there, and their frequencies add up. □

Proof of Prop. 11. The projected constraints of a schema S over R imply exactly the FDs over S that $\Sigma[E]$ implies, so the atomic closure of S consists of the members of $\mathcal{A}$ inside S. A member $X \to a$ inside S therefore lies inside S', and if X is no superkey of S, it is no superkey of $S' \supseteq S$. Hence every non-key atomic FD of S is a non-key atomic FD of S', and since heats are sums of positive numbers over these sets, $\mathcal{H}_\theta(S) \leq \mathcal{H}_\theta(S')$. □

Proof of Prop. 13. A schema (R_i, Σ_i) is in E-BCNF if and only if every member of $\mathcal{A}_i$ has a superkey determinant, since every non-trivial implied FD has a member of $\mathcal{A}_i$ with the same right-hand side and a determinant inside its own. This holds if and only if the sum in Def. 9 is empty. An empty sum is 0, and a non-empty one is positive, since θ_E takes positive values. Since E-BCNF implies E-3NF, heat 0 is the same as $(0, E)$-TNF. □

Proof of Prop. 14. The nesting is immediate, and the heat of any schema is at most $\mathcal{H}_{\text{tot}}$, so the top level imposes no bound beyond E-3NF. For strictness between levels $h < h'$, take $R = \{a, b, c\}$ with $\Sigma = \{(R;\, ab \to c),\ (R;\, c \to b)\}$, scope $E = R$, and $\theta(b) = h'$. The minimal keys are ab and ac, so every attribute is prime and the schema is in E-3NF. Its atomic closure is $\{ab \to c,\ c \to b\}$, of which only $c \to b$ is non-key, so $\mathcal{H}_\theta(R) = h'$. Hence the schema is in (h', E)-TNF but not in (h, E)-TNF. □

Proof of Prop. 15. Under constant $\theta_E \equiv w$ the heat of every schema is w times its number of non-key atomic FDs, and maximizing or minimizing over subschemata scales by the constant w. □

Proof of Prop. 16. In the delivery schema of Ex. 1, set the heat of the courier c to $2L$ and of every other attribute to 1. The atomic closure and the minimal keys of R are as listed in Ex. 10. Each decomposition of Fig. 1 has six subschemata and leaves eight of the 18 members of $\mathcal{A}$ unembedded, all implied by the members it embeds. Seven of them have two-attribute determinants, such as $bc \to g$, implied by $bc \to v$ and $v \to g$, and $rz \to b$, implied by $rz \to v$ and $vz \to b$. The eighth is a zone rule: $bdv \to z$ lies in no subschema of $\mathcal{D}_f$ and is implied by $bv \to c$ and $bcd \to z$, and $bcd \to z$ lies in none of $\mathcal{D}_\theta$ and is implied by $bc \to v$ and $bdv \to z$. Both decompositions are therefore E-dependency-preserving, and both are E-lossless by the classical synthesis argument on the scope [29], since each contains a minimal key of R, namely $bcd \subseteq bcdz$ and $bdz \subseteq bdvz$. Five subschemata per decomposition are in E-BCNF. The remaining ones are $bcdz$, with the non-key FD $z \to c$, and $bdvz$, with the non-key FDs $bz \to v$ and $vz \to b$ (Ex. 10), and both are in E-3NF, since c, v, and b lie in the minimal keys bcd and bdv of their subschemata. Hence $\mathcal{D}_f$ materializes one FD redundantly and $\mathcal{D}_\theta$ two. None of this depends on θ, so it holds for every L. The heats of $\mathcal{D}_f$ and $\mathcal{D}_\theta$ are $2L$ and 2, so the design that stores the single non-key FD is the hotter one, and the separation is the factor L. □

Proof of Prop. 19. Suppose some $X \to a \in \mathcal{A}_i$ is non-key in R_i, and let X^+ be the closure of X in (R_i, Σ_i). Take the two-tuple instance whose tuples agree exactly on X^+ and differ elsewhere. The tuples are distinct because X is not a superkey. This instance satisfies Σ_i, since whenever both tuples agree on a determinant Y of an implied FD $Y \to b$, we have $Y \subseteq X^+$, hence $b \in X^+$, so they agree on b. In particular, no minimal key lies inside X^+, as it would make X a superkey. Applying the mode $X \to a$ with a fresh value to one tuple alone now violates $X \to a$ itself, while rewriting both tuples restores agreement on X^+ and hence satisfaction, so the application is anomalous.

Conversely, let the schema be in E-BCNF, so every determinant of $\mathcal{A}_i$ is a superkey of R_i, and two distinct tuples of an instance $r_i \models \Sigma_i$ never agree on one. Consider an application of any mode $X \to A$, whether Σ_i implies $X \to A$ or not, and rewrite the A-value of a single tuple of its X-group to the fresh value. The rewritten tuple then agrees with no other tuple on a set that contains A, and its agreements on the other sets are unchanged, so still no two distinct tuples agree on a determinant of $\mathcal{A}_i$. Hence every member of $\mathcal{A}_i$, and with them Σ_i, still holds, and the application is not anomalous. □

Proof of Thm. 20. For hardness, let θ_E be constant with value c and let $h = c \cdot |R_i| \cdot 2^{|R_i|}$, a number of polynomially many bits. Since $|\mathcal{A}_i| \leq |R_i| \cdot 2^{|R_i|}$, the bound h is at least the total heat of $\mathcal{A}_i$, so membership coincides with E-3NF (Prop. 14).

E-3NF is 3NF of the projected constraints, which is NP-hard to recognize by the classical primality argument [17], [27].

Now let $\mathcal{A}_i$ be computable in polynomial time. Then $\mathcal{H}_\theta(R_i)$ is computed by one attribute closure per member of $\mathcal{A}_i$, which decides whether its determinant is a superkey of R_i, followed by a sum.

For membership in NP, the verifier computes $\mathcal{A}_i$ and $\mathcal{H}_\theta(R_i)$ as above and compares the heat against h. It remains to verify E-3NF, which holds if and only if every non-key member of $\mathcal{A}_i$ has a prime right-hand side, since every non-trivial implied FD has a member with the same right-hand side and a determinant inside its own. The certificate supplies, for every member $X \to a$ whose determinant an attribute closure shows to be no superkey, a minimal key of R_i containing a. Each supplied key is verified to be a key by one closure and to be minimal by dropping each attribute in turn. The verifier accepts if and only if every such right-hand side lies in a verified minimal key, and such keys exist exactly when the schema is in E-3NF, so verification is sound and complete.

For the interreducibility, E-3NF reduces to determination on the same input by taking θ_E constant and h the bound above. Conversely, determination reduces to one E-3NF test followed by the polynomial heat computation and the comparison against h.

For the design problem, let θ_E be constant with value 1 and $h = 0$. A schema has heat 0 if and only if none of its atomic FDs is non-key, that is, if and only if it is in E-BCNF (Prop. 13), so the question is whether an E-lossless, E-dependency-preserving decomposition into E-BCNF exists. For $E = R$ this is the existence of a lossless, dependency-preserving decomposition into BCNF, which is NP-hard [19]. Losslessness does not change the question, since adding a minimal key to a dependency-preserving decomposition into BCNF makes it lossless and keeps it in BCNF, as a minimal key carries no non-trivial FD. □

Proof of Lem. 21. Let $\mathcal{D}$ be such a decomposition, and let $F^{\mathcal{D}}$ collect the members of $\mathcal{A}$ whose attributes lie inside some subschema of $\mathcal{D}$. Every non-trivial implied FD $Y \to a$ whose attributes lie inside a subschema has a member $X \to a$ of $\mathcal{A}$ with $X \subseteq Y$, which lies inside that subschema too. Hence the projected constraints of the subschemata imply exactly what $F^{\mathcal{D}}$ implies, and E-dependency-preservation yields that $F^{\mathcal{D}}$ implies $\mathcal{A}$. Every $X \to a \in F^{\mathcal{D}}$ lies inside some subschema R_i of $\mathcal{D}$, so $\mathcal{H}_\theta(R_i) \geq \mathcal{H}_\theta(Xa)$ by Prop. 11. Taking maxima, the maximal subschema heat of $\mathcal{D}$ is at least $\max_{X \to a \in F^{\mathcal{D}}} \mathcal{H}_\theta(Xa)$, which is at least h^* since $F^{\mathcal{D}}$ is one of the sets the minimum ranges over. □

Proof of Thm. 23. The loop drops an atomic FD only when the surviving working set still implies it, so $\mathcal{A}_w$ remains a cover of $\mathcal{A}$ throughout. Every member of the final $\mathcal{A}_w$ is embedded in the subschema built for it in line 3, so the output is E-dependency-preserving. Line 4 removes only subschemata inside others, which keeps their FDs embedded and cannot raise the maximal heat. Each (Xa, Σ_{Xa}) is in E-3NF, by the classical argument applied to the reduct. For a key subschema added in line 6, a minimal key K admits no implied FD $Y \to b$ with $Yb \subseteq K$ and $b \notin Y$, since $K \setminus \{b\}$ would then determine b and hence all of R, contradicting minimality. That subschema therefore carries no non-trivial constraint, is in E-BCNF, and has heat 0. The output is E-lossless, since it is E-dependency-preserving and, after line 6, some subschema is a superkey of $(R, \Sigma[E])$, either one retained from line 4 or the added minimal key. The classical synthesis argument thus applies to the scope, a total relation satisfying $\Sigma[E]$ [29], [70], and the tuples outside the scope are stored in the full-width remainder [17].

For optimality, let $\mathcal{D}'$ be any decomposition obtained from $\mathcal{A}$ by eliminating redundant atomic FDs, and let h' be its maximal subschema heat. Such a decomposition carries one subschema Xa per retained FD $X \to a$, so h' is the largest heat of these schemata. Suppose the output kept a subschema (Xa, Σ_{Xa}) with $\mathcal{H}_\theta(Xa) > h'$. Since $\mathcal{D}'$ has no subschema of heat above h', it eliminated $X \to a$, so the members of $\mathcal{A}$ that $\mathcal{D}'$ retains imply it, and each of them has heat at most h'. A positive heat makes (Xa, Σ_{Xa}) critical (Prop. 13), so line 3 visits $X \to a$ before every FD whose schema is cooler, and at that moment $\mathcal{A}_w$ still contains every FD whose schema has heat at most h' and hence implies $X \to a$. The test succeeds and removes $X \to a$, contradicting its presence in the output. The maximal subschema heat of the output is thus at most h' for every such $\mathcal{D}'$. Every $F \subseteq \mathcal{A}$ that implies $\mathcal{A}$ is the retained set of such a decomposition, since eliminating the members of $\mathcal{A} \setminus F$ in any order succeeds while F is retained, and that decomposition has maximal subschema heat $\max_{X \to a \in F} \mathcal{H}_\theta(Xa)$. Hence the output has maximal subschema heat at most h^*, and at least h^* by Lem. 21.

The secondary claim repeats the argument on the E-BCNF schemata, which the loop examines last, with the refresh weight in place of the heat. Let $\mathcal{C}$ be the critical members the loop retains, and let the cost of an E-BCNF member $X \to a$ be 0 if Xa lies inside the schema of a member of $\mathcal{C}$ and $w_\theta(Xa)$ otherwise, the number line 2 orders the E-BCNF schemata by. Let $\mathcal{D}'$ be obtained by eliminating redundant atomic FDs from the working set $\mathcal{A}_w$ left once every critical schema has been examined, and let n' be the maximal refresh weight over the E-BCNF subschemata that remain after the subsumption step. Every critical member of $\mathcal{A}_w$ was irredundant in a superset of $\mathcal{A}_w$, so $\mathcal{D}'$ retains it, and the critical members of $\mathcal{D}'$ are those of $\mathcal{C}$. Every E-BCNF member that $\mathcal{D}'$ retains therefore costs at most n': its schema lies inside the schema of a member of $\mathcal{C}$, or it is or lies inside an E-BCNF subschema of $\mathcal{D}'$, whose refresh weight is at least its own. Suppose the output kept an E-BCNF subschema (Xa, Σ_{Xa}) of refresh weight above n'. No schema of a member of $\mathcal{C}$ contains Xa, since line 4 would have removed it, so $X \to a$ costs $w_\theta(Xa) > n'$, and $\mathcal{D}'$ eliminated it. The FDs $\mathcal{D}'$ retains thus imply $X \to a$, and each of them is in $\mathcal{C}$, which the loop has already retained, or costs at most n', so that the loop examines it after $X \to a$. All of them are therefore still in $\mathcal{A}_w$ when the test on $X \to a$ runs, so the test succeeds and removes $X \to a$, contradicting its presence in the output. □

Proof of Prop. 25. The atomic closure of a schema Xa consists of the members of $\mathcal{A}$ inside Xa (proof of Prop. 11), so the E-BCNF test and the heat of a member of $\mathcal{A}$ each take one attribute closure per member inside Xa. The loop performs one implication test per member of $\mathcal{A}$, each an attribute closure, and orders the members by heat and by sums of θ over at most $|R|$ attributes. Line 4 is a pairwise containment scan, the test before line 6 takes one attribute closure per subschema, and line 6 computes one minimal key by removing attributes from R while the remaining ones still determine R. All steps are polynomial in $|\mathcal{A}|$ and $|R|$. □

Proof of Thm. 26. Every schema S has heat $\sum_b n_S(b)\,\theta(b)$, since θ_E gives each member the heat of its right-hand side. By the proof of Prop. 11, a schema dominates each of its subsets, and a dominating schema is no cooler, since heats are positive. Fix a heat map and a cover F with $\max_{X\to a\in F} \mathcal{H}_\theta(Xa) = h^*$ (Lem. 21). For (1), every subschema S of $\mathcal{D}$ is dominated by the schema Xa of some member of F, so $\mathcal{H}_\theta(S) \le \mathcal{H}_\theta(Xa) \le h^*$. Hence $\mathcal{D}$ has heat at most h^*, and at least h^* by Lem. 21. For the first claim of (2), let the cover F' avoid an attribute b that $\mathcal{D}$ stores, and let M be the largest number of non-key atomic FDs of a schema Xa with $X \to a \in F'$. Declare b at heat $LM + 1$ and leave every other attribute at the default heat 1. None of the non-key atomic FDs of such an Xa determines b, so $\mathcal{H}_\theta(Xa) \le M$, and $h^* \le M$ by the definition of h^*. The subschema of $\mathcal{D}$ that stores b has heat at least $LM + 1$, which exceeds $L \cdot h^*$. Otherwise, every cover contains, for each attribute b that $\mathcal{D}$ stores, a member whose schema stores b. This holds for F in particular, so $h^* \ge \theta(b)$ for each such b. A subschema S of $\mathcal{D}$ therefore has heat $\sum_b n_S(b)\,\theta(b) \le N \cdot \max_b \theta(b) \le N \cdot h^*$, the maximum taken over the attributes that S stores. □

Proof of Prop. 28. Under newcomer-wins, after the entry all stored tuples of the group must carry $t(A)$. A subschema on which $X \to A$ is non-key stores, for the group, one row per distinct projection of the group's tuples, each carrying the old A-value, and all of them must be rewritten. If the subschema separates the group's tuples, that is, if these projections are pairwise distinct, the count is k. If every subschema containing XA has X as a key, each of them stores the group in a single row, and rewriting that one row in each restores $X \to A$. The other rules that determine A carry their groups along (Lem. 6), again one row per group wherever such a rule is stored under a key. □

Proof of Prop. 30. (1) A retraction moves no tuple into the scope, so no pending tuple gains a scope tuple to disagree with. If the retracted attribute belongs to XA, the retracted tuple is no longer XA-total and carries no latent conflict with $X \to A$. Otherwise its XA-values are unchanged, and a retraction that moves it out of the scope leaves it agreeing on A with its X-group, since r satisfies the constraint there. (2) The mode rewrites every tuple of its scope $r^{E'}$ in the group, pending or not, and these keep agreeing with the moved group. An XA-total pending tuple that agreed with the group but is not E'-total keeps the old A-value, so once the group moves to a fresh value, it disagrees with the scope and carries a latent conflict. (3) Refreshes, retractions, and inserts outside the scope never move a tuple into the scope, and a tuple inserted into the scope has no pending past, so a latent conflict can surface only through the completion that makes its carrier E-total, whose writes Prop. 28 counts. (4) A tuple inserted into the scope is E-total, so every constraint of the scope applies to it at once, and Prop. 28 counts the writes of the newcomer-wins repair. A tuple inserted outside the scope is invisible to the constraints of the scope, so it violates none, and its values are as inserted, so Def. 29 applies exactly when it is XA-total and disagrees on A with its X-group. □